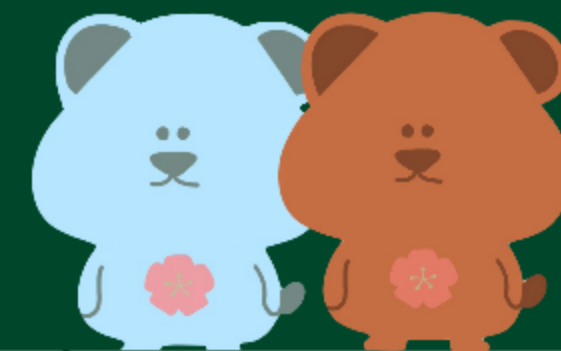




회귀

2팀 한송희 허정인 이재린

목차

#01 회귀 소개, 단순 선형 회귀, 경사 하강법

#02 실습 - 보스턴 주택 가격 예측, 다항 회귀

#03 규제 선형 회귀 - 릿지, 라쏘, 엘라스틱넷, 로지스틱 회귀, 회귀 트리



회귀분석



#회귀분석

#5.1 회귀 소개

#5.2 단순 선형 회귀

#5.3 경사 하강법

#5.1 회귀 소개

회귀(Regression): 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향

→ 회귀분석: 이러한 회귀를 이용한 통계학 기법

여러 개의 독립변수+한 개의 종속변수: 이들 간의 상관관계를 모델링

#5.1 회귀 소개

e.g. 아파트 가격 예측

선형 회귀식: $Y = W_1 * X_1 + W_2 * X_2 + W_3 * X_3 + \dots + W_n * X_n$

Y : 종속변수=아파트 가격=결정 값

$X_1, X_2, X_3, \dots, X_n$: 독립변수=방 개수, 방 크기, 주변 학군...=feature

$W_1, W_2, W_3, \dots, W_n$: 독립변수 감성 영향을 미치는 회귀계수(Regression coefficients)

머신러닝 회귀 예측의 핵심
=feature와 결정 값 데이터 기반 학습을 통해 최적의 회귀
계수를 찾기

#5.1 회귀 소개

-회귀 유형

독립변수 개수	회귀 계수의 결합
1개: 단일회귀	선형: 선형 회귀
여러 개: 다중 회귀	비선형: 비선형 회귀

-지도학습 유형

Classification → category 값(이산값)

Regression → 숫자값(연속값)

#5.1 회귀 소개

선형 회귀: 실제 값과 예측값의 차이(RSS)를 최소화 하는 직선형 회귀선을 최적화

→ 여기에 Regularization방법에 따라 유형이 나뉨

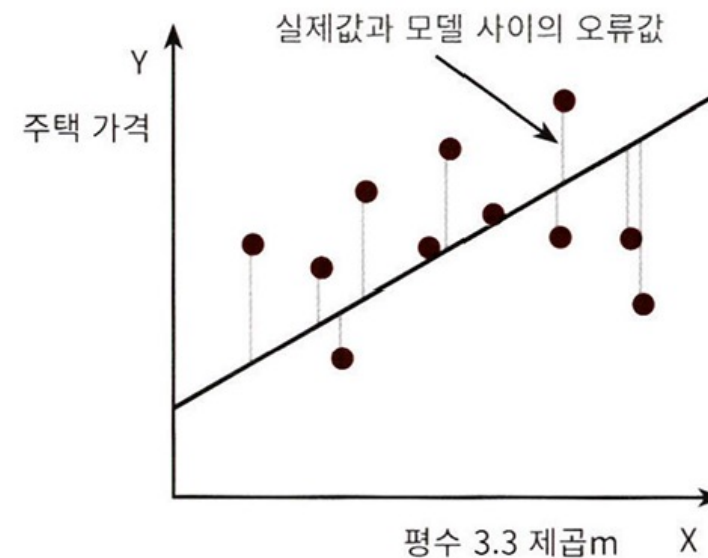
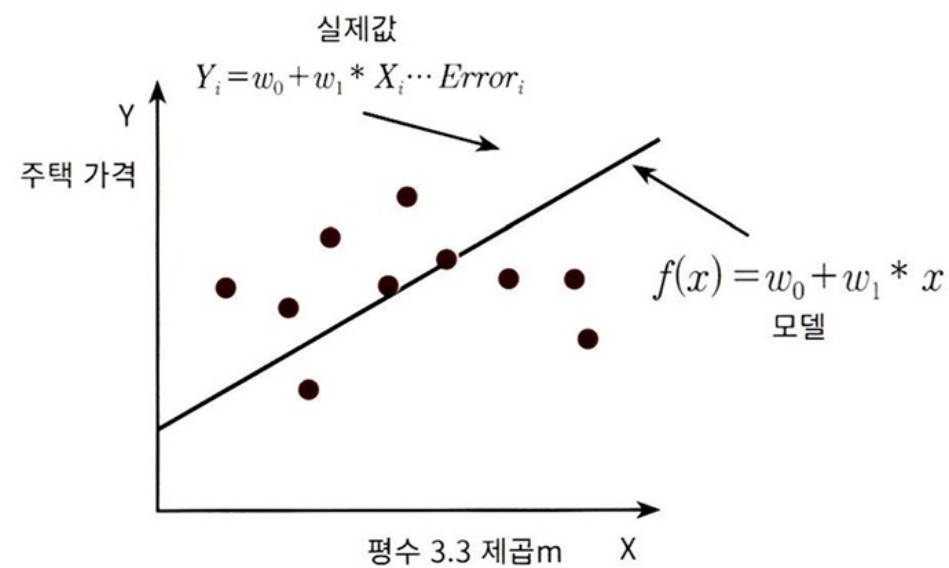
*Regularization: 과적합 문제를 해결하기 위해 회귀 계수에 페널티 값 적용

종류	특징
일반선형회귀	Regularization 적용 안함
Ridge	L2 Regularization:상대적으로 큰 회귀 계수 값의 예측 정확도를 감소시키기 위해 회귀 계수값을 더 작게 만든다
Lasso	L1 Regularization: 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측시 피처가 선택되지 않도록 한다
ElasticNet	L1+L2: 피처가 많은 데이터세트에서 L1으로 피처의 개수를 줄이는 동시에 L2로 규제 계수 값 크기를 조절한다
Logistic Regression	Classification에 사용되는 선형모델. 강력한 분류 알고리즘

#5.2 단순 선형 회귀

단순 선형 회귀: 독립 변수 하나, 종속 변수 하나

e.g. 주택크기만으로 결정되는 주택 가격



→ 직선 형태의 그래프(선형)

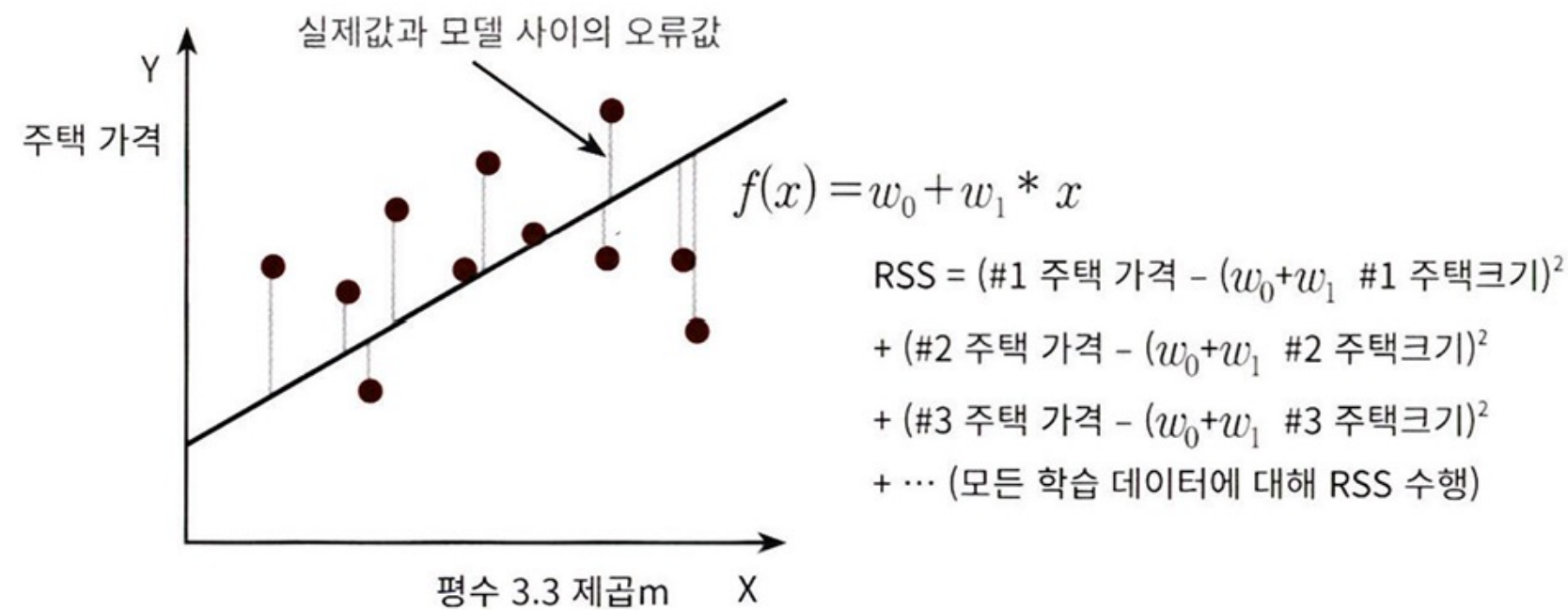
→ 잔차(오류값)

최적의 회귀 모델=잔차의 합이 최소가 되는 모델=합이 최소가 되는 회귀 계수 찾기

#5.2 단순 선형 회귀

-오류값의 합을 구하는 방법

- 1) Mean Absolute Error:절댓값을 취해서 더하기
- 2) **RSS: 제곱을 구해서 더하기**



w_0 : 절편
 w_1 : 기울기

RSS는 w 변수(회귀계수)가 중심 변수!!!

비용 함수=손실함수(loss function)

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i 는 1부터 학습 데이터의 총 건수 N 까지)

RSS: 비용(Cost)
 w : 변수(회귀계수)

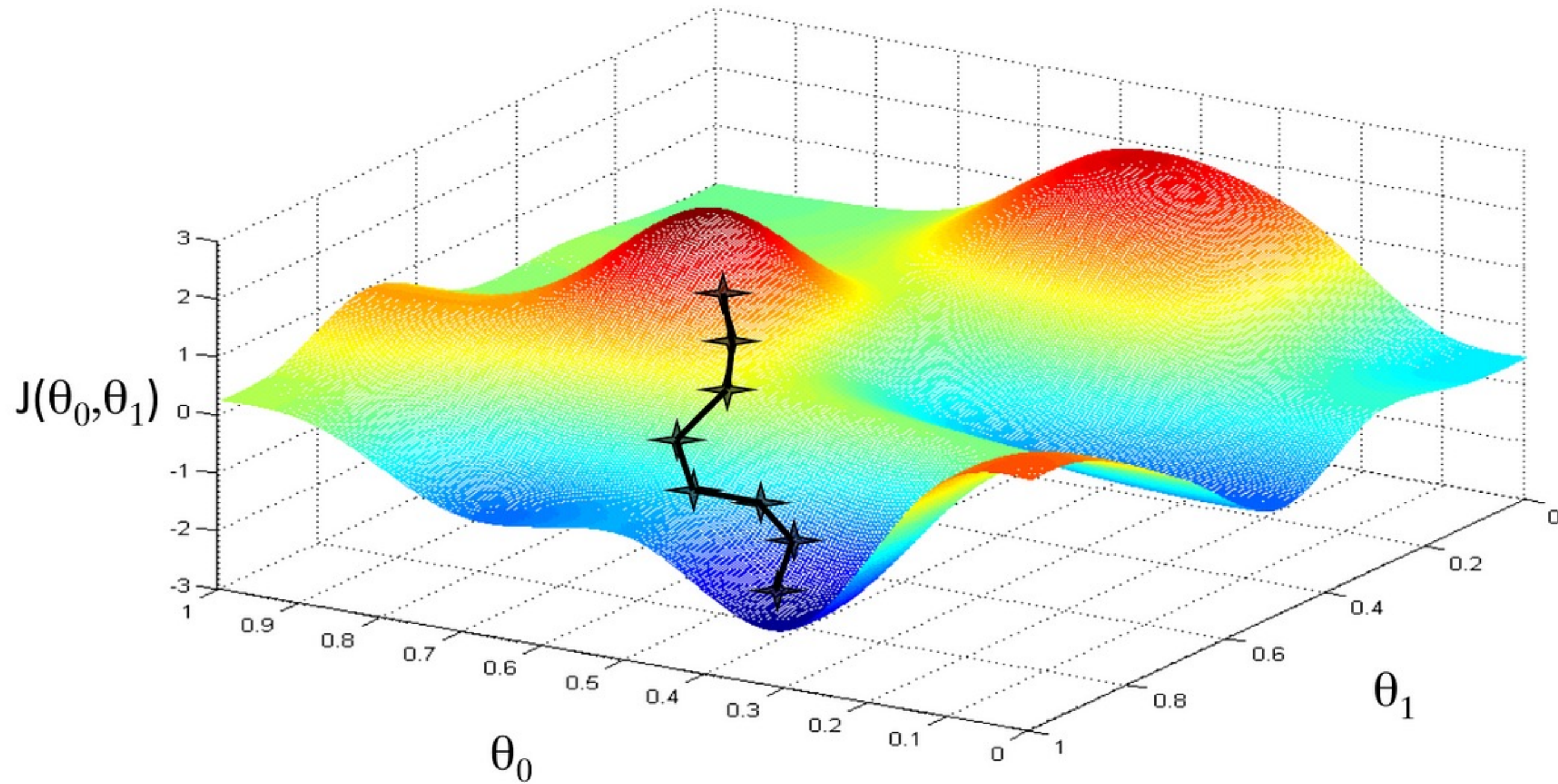
#5.3 경사 하강법

경사 하강법(Gradient Descent): 비용함수(RSS)가 최소가 되는 w 파라미터를 구하는 방법

‘점진적으로’ 반복적인 계산을 통해 w 파라미터의 값을 업데이트하며 오류 값이 최소가 되도록 하는 w 파라미터를 구함

→ 머신러닝: ‘데이터를 기반으로 알고리즘이 스스로 학습’을 실현

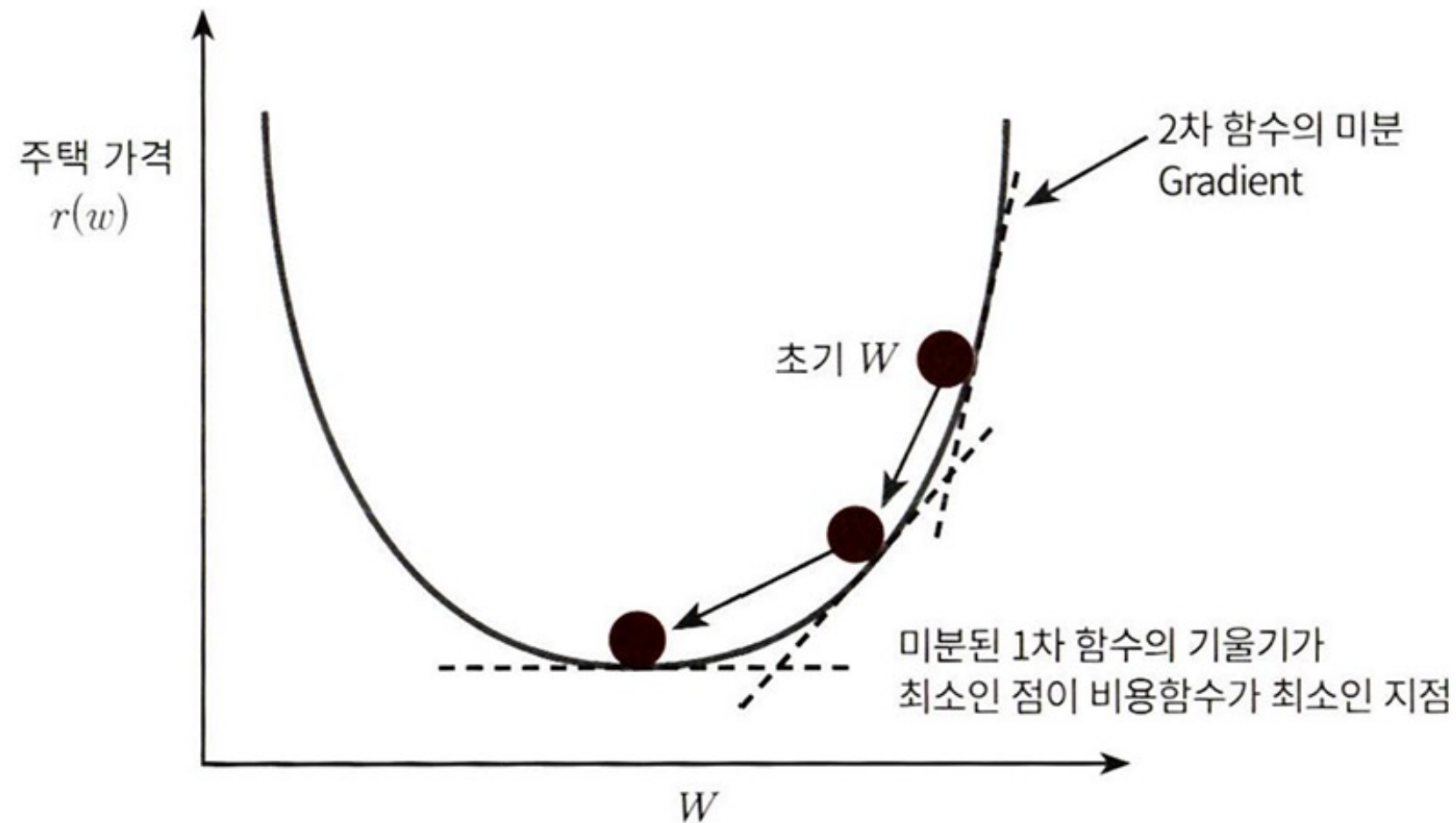
#5.3 경사 하강법



→ 현재 위치보다 낮은 곳으로 계속 이동

#5.3 경사 하강법

어떻게 하면 오류가 작아지는 방향으로 w 값을 보정할 수 있는가?



최초 w 에서 미분 적용 → 미분 값이 감소하는 방향으로 w 업데이트 → 미분된 1차함수 기울기가 감소하지 않는 지점 = 비용함수가 최소인 지점 → w 반환

#5.3 경사 하강법

-수식

$$\text{RSS}(w_0, w_1) = R(w) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

w1과 w0 각각에 대해 편미분:

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

→ 이들을 반복적으로 보정하면서 w1, w0 값을 업데이트해 R(w)가 최소가 되는 w1, w0를 구할 수 있다.

$$\text{새로운 } w_1 = \text{이전 } w_1 + \eta \frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i) \quad \text{새로운 } w_0 = \text{이전 } w_0 + \eta \frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

#5.3 경사 하강법

Step1: w_1, w_0 를 임의의 값으로 설정하고 첫 비용함수 계산



Step2: w_1, w_0 를 새로운 값으로 업데이트 한 후 다시 비용함수 계산



Step3: 비용 함수 값이 감소하지 않을 때까지 반복



최종 w_1, w_0 반환

#5.3 경사 하강법

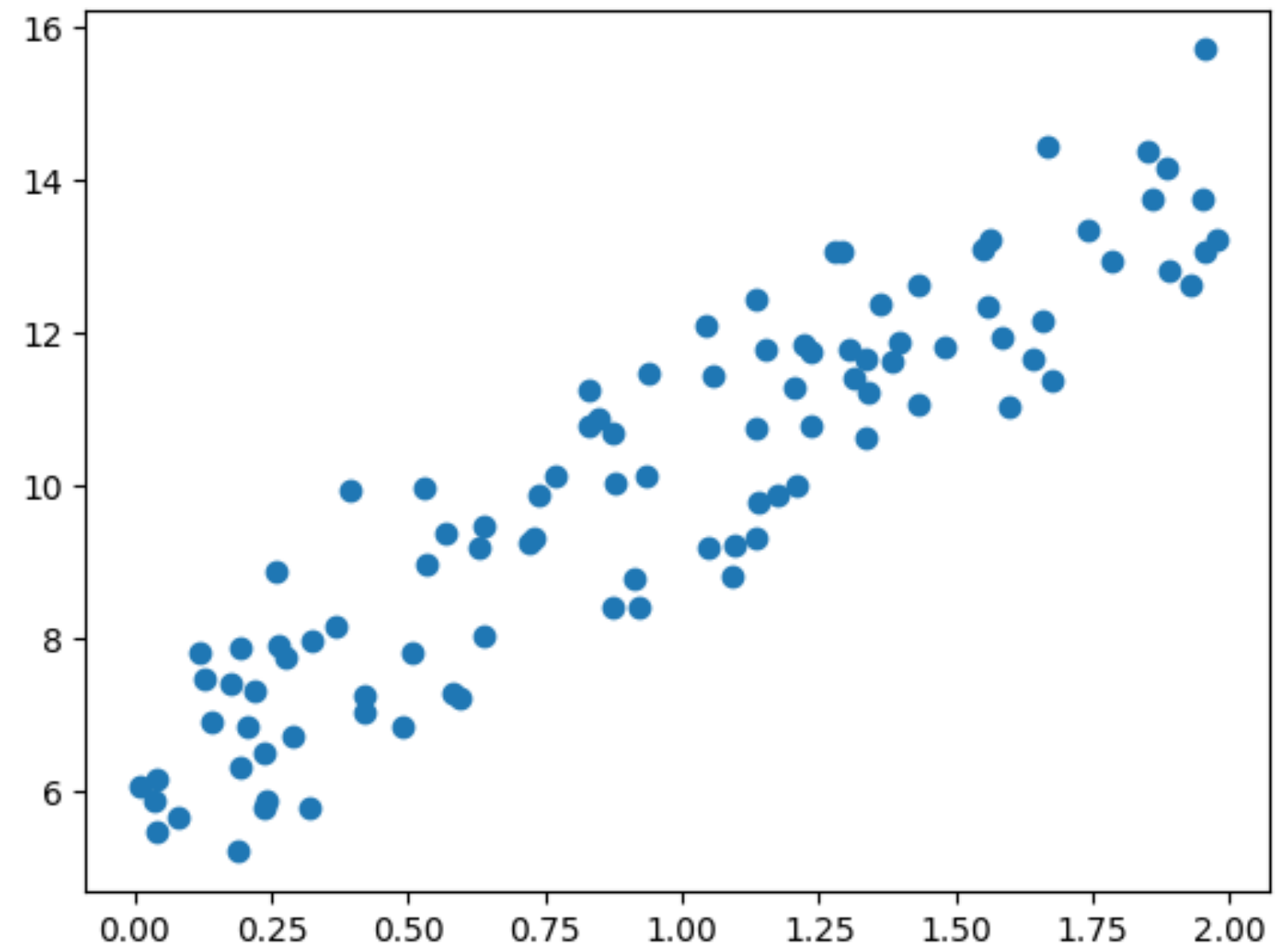
$y=4X+6$ 근사하기: 100개의 데이터 세트를 경사하강법을 이용해 회귀 계수 w_1, w_0 도출

1) 데이터 생성+시각화

```
[1] import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(0)
#y=4X+6을 근사(w1=4, w0=6), 임의의 값은 노이즈를 위해 만듦
X=2*np.random.rand(100,1)
y=6+4*X+np.random.randn(100,1)

#X,y 데이터 세트 산점도로 시각화
plt.scatter(X,y)
```



#5.3 경사 하강법

2) 비용함수(get_cost) 정의하기

```
def get_cost(y, y_pred):  
    N=len(y)  
    cost=np.sum(np.square(y-y_pred))/N  
    return cost
```

#5.3 경사 하강법

2) 경사하강법(gradient_descent) 정의하기

```
#w1과 w0를 업데이트 할 w1_update, w0_update를 반환
def get_weight_updates(w1,w0,X,y,learning_rate=0.01):
    N=len(y)

    #w1_update,w0_update를 각각 w1,w0의 shape와 동일한 크기를 가진 0값으로 초기화
    w1_update=np.zeros_like(w1)
    w0_update=np.zeros_like(w0)
    #예측 배열 계산하고 예측과 실제 값의 차이 계산
    y_pred=np.dot(X,w1.T)+w0
    diff=y-y_pred

    #w0_update를 dot행렬 연산으로 구하기 위해 모두 1값을 가진 행렬 생성
    w0_factors=np.ones((N,1))
    #w1과 w0을 업데이트할 w1_update와 w0_update 계산
    w1_update=-(2/N)*learning_rate*(np.dot(X.T,diff))
    w0_update=-(2/N)*learning_rate*(np.dot(w0_factors.T,diff))

    return w1_update, w0_update
```

#5.3 경사 하강법

2) 경사하강법(gradient_descent) 정의하기

```
#입력 인자 iters로 주어진 횟수만큼 반복적으로 w1과 w0를 업데이트를 적용
def gradient_descent_steps(X,y,iters=10000):
    #w0와 w1을 모두 0으로 초기화
    w0=np.zeros((1,1))
    w1=np.zeros((1,1))

    #인자로 주어진 iters만큼 반복적으로 get_weight_updates()를 호출해 w1,w0 업데이트 수행
    for ind in range(iters):
        w1_update,w0_update=get_weight_updates(w1,w0,X,y,learning_rate=0.01)
        w1=w1-w1_update
        w0=w0-w0_update

    return w1,w0
```

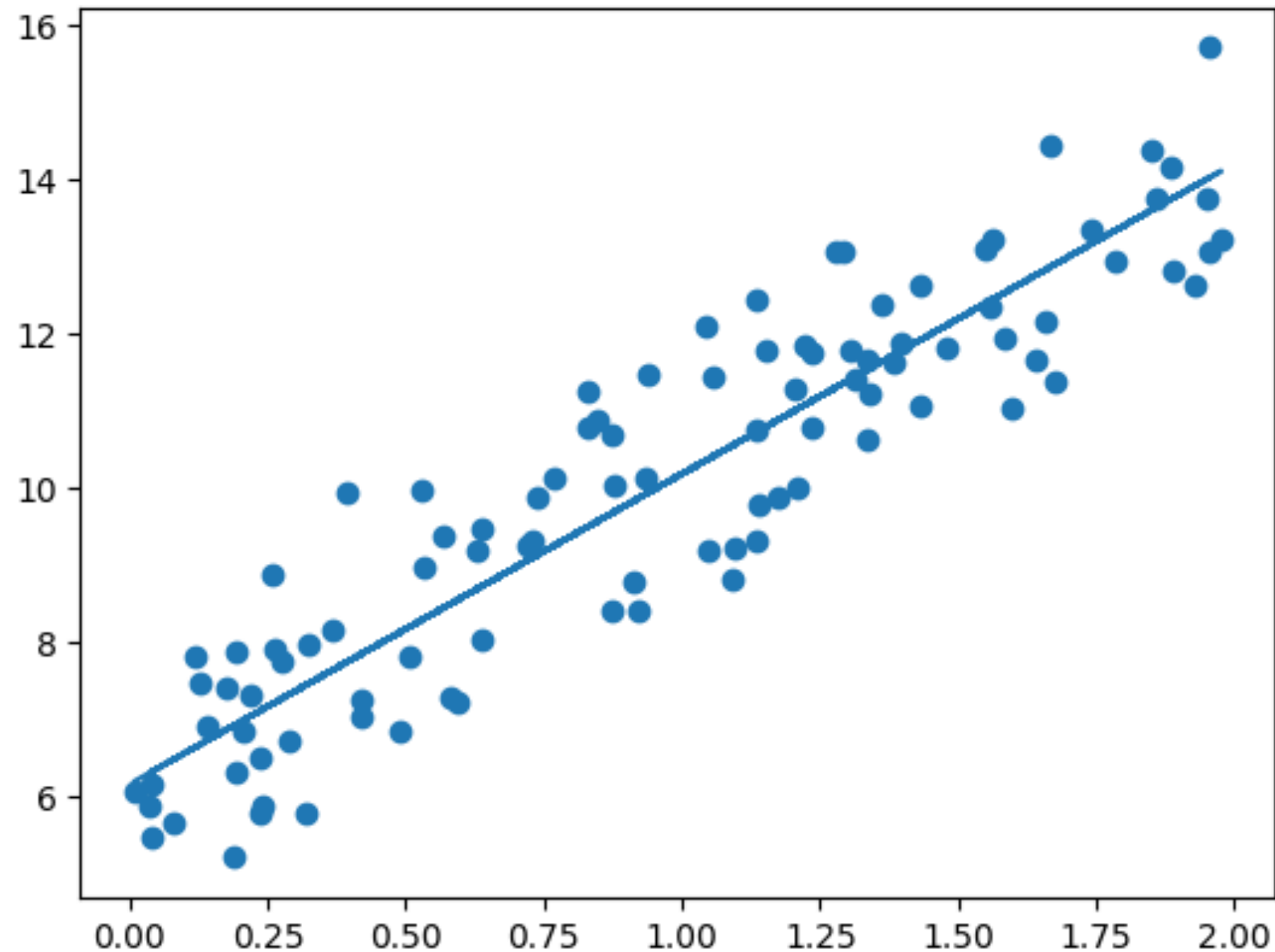
```
w1,w0=gradient_descent_steps(X,y,iters=1000)
print('W1:{0:.3f} w0:{1:.3f}'.format(w1[0,0],w0[0,0]))
y_pred=w1[0,0]*X+w0
print('Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y,y_pred)))
```

```
W1:4.022 w0:6.162
Gradient Descent Total Cost:0.9935
```

#5.3 경사 하강법

3) 회귀선 그리기

```
plt.scatter(X,y)  
plt.plot(X,y_pred)
```



경사하강법 → 수행시간이 매우 오래 걸림



확률적 경사
하강법(Stochastic Gradient
Descent)을 이용

#5.3 경사 하강법

+) 확률적 경사 하강법(Stochastic Gradient Descent)

```
def stochastic_gradient_descent_steps(X,y,batch_size=10, iters=1000):  
    w0=np.zeros((1,1))  
    w1=np.zeros((1,1))  
    prev_cost=100000  
    iter_index=0  
  
    for ind in range(iters):  
        np.random.seed(ind)  
        #전체 X,y 데이터에서 랜덤하게 batch_size만큼 데이터를 추출해 sample_X, sample_y로 저장  
        stochastic_random_index=np.random.permutation(X.shape[0])  
        sample_X=X[stochastic_random_index[0:batch_size]]  
        sample_y=y[stochastic_random_index[0:batch_size]]  
        #랜덤하게 batch_size만큼 추출된 데이터 기반으로 w1_update,w0_update 계산 후 업데이트  
        w1_update,w0_update=get_weight_updates(w1,w0,sample_X,sample_y, learning_rate=0.01)  
        w1=w1-w1_update  
        w0=w0-w0_update  
  
    return w1,w0
```

```
w1,w0=stochastic_gradient_descent_steps(X,y, iters=1000)  
print('w1:',round(w1[0,0],3), 'w0:',round(w0[0,0],3))  
y_pred=w1[0,0]*X+w0  
print('Stochastic Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y,y_pred)))
```

```
w1: 4.028 w0: 6.156  
Stochastic Gradient Descent Total Cost:0.9937
```

#5.3 경사 하강법

If 피처가 여러 개?

예측값: $\hat{Y} = w_0 + w_1 * X$

피처가 M개 → 회귀 계수 M+1 (w0)개

$$\hat{Y} = w_0 + w_1 * X_1 + w_2 * X_2 + \dots + w_{100} * X_{100}$$

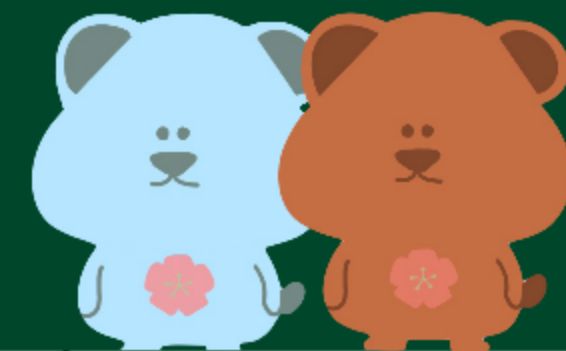
$\hat{Y}$ 1값을 가진 피처 추가 X_{mat}

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix} = \begin{bmatrix} \text{Feat 0} & \text{Feat 1} & \text{Feat 2} & \dots & \text{Feat M} \\ 1 & x_{11} & x_{12} & \dots & x_{1m} \\ 1 & x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix}$$

w_0 을 W 배열 내에 포함

내적 $\star \begin{bmatrix} w_0 & w_1 & w_2 & \dots & w_m \end{bmatrix}^T$

$$\hat{Y} = X_{mat} * W^T$$



5-4~5-5. 실습과 다항회귀

5.4 주택 가격 예측



#5.4.1.LinearRegression 클래스-Ordinary Least Squares

LinearRegression 클래스

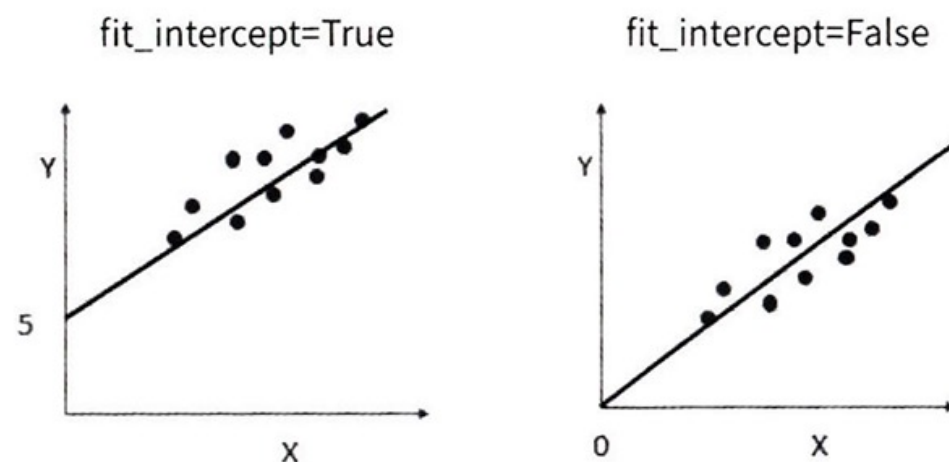
- 예측값과 실제 값의 RSS를 최소화해 OLS(Ordinary Least Squares) 추정 방식으로 구현한 클래스

```
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=False, copy_X=True, n_jobs=1)
```

W를 coef_속성에 저장

fit_intercept: 불린 값으로, 디폴트는 True입니다. Intercept(절편) 값을 계산할 것인지 말지를 지정합니다. 만일 False로 지정하면 intercept가 사용되지 않고 0으로 지정됩니다.

입력 파라미터



normalize: 불린 값으로 디폴트는 False입니다. fit_intercept가 False인 경우에는 이 파라미터가 무시됩니다. 만일 True이면 회귀를 수행하기 전에 입력 데이터 세트를 정규화합니다.

속성

coef_: fit() 메서드를 수행했을 때 회귀 계수가 배열 형태로 저장하는 속성. Shape는 (Target 값 개수, 피쳐 개수).

intercept_: intercept 값

OLS기반의 회귀 계수 계산

=> 다중 공선성 문제 주의

#5.4.2. 회귀 평가 지표

회귀 평가 지표

- 회귀의 평가를 위한 지표는 실제 값과 회귀 예측값의 차이 값을 기반으로 한 지표가 중심
- 실제값과 예측값의 차이를 그냥 더하면 +와 -가 섞여서 오류가 상쇄
⇒ 오류의 절댓값 평균이나 제곱, 또는 제곱한 뒤 다시 루트를 씌운 평균값을 구함

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$

* 이밖에 MSE나 RMSE에 로그를 적용한 MSLE와 RMSLE도 사용

#5.4.2. 회귀 평가 지표

- 사이킷런은 RMSE 제공 X
⇒ MSE에 제곱근을 씌워서 계산하는 함수를 직접 만들어야 함

평가 방법	사이킷런 평가 지표 API	Scoring 함수 적용 값
MAE	<code>metrics.mean_absolute_error</code>	<code>'neg_mean_absolute_error'</code>
MSE	<code>metrics.mean_squared_error</code>	<code>'neg_mean_squared_error'</code>
R^2	<code>metrics.r2_score</code>	<code>'r2'</code>

- 'neg_' 접두어: 음수 값을 의미
- 음수 반환 이유
 - 사이킷런의 Scoring 함수는 점수가 클수록 좋은 평가
 - 오류가 클수록 모델 성능이 나빠짐
- 해결 방법 :
 - MAE 값에 -1을 곱해 음수로 변환
 - 작은 오류가 더 큰 점수로 인식되도록 조정

#5.4.3. LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

```
# 필요한 라이브러리 설치
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

from scipy import stats

# boston 데이터셋 준비
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

# 데이터 세트 DataFrame 변환
bostonDF = pd.DataFrame(data, columns=['CRIM', 'ZN', 'INDUS', 'CHAS',
    'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT'])

# 데이터 세트의 target 배열은 주택 가격임.
# 이를 PRICE 칼럼으로 DataFrame에 추가함.

bostonDF['PRICE'] = target
print('Boston 데이터 세트 크기: ', bostonDF.shape)
bostonDF.head()
```

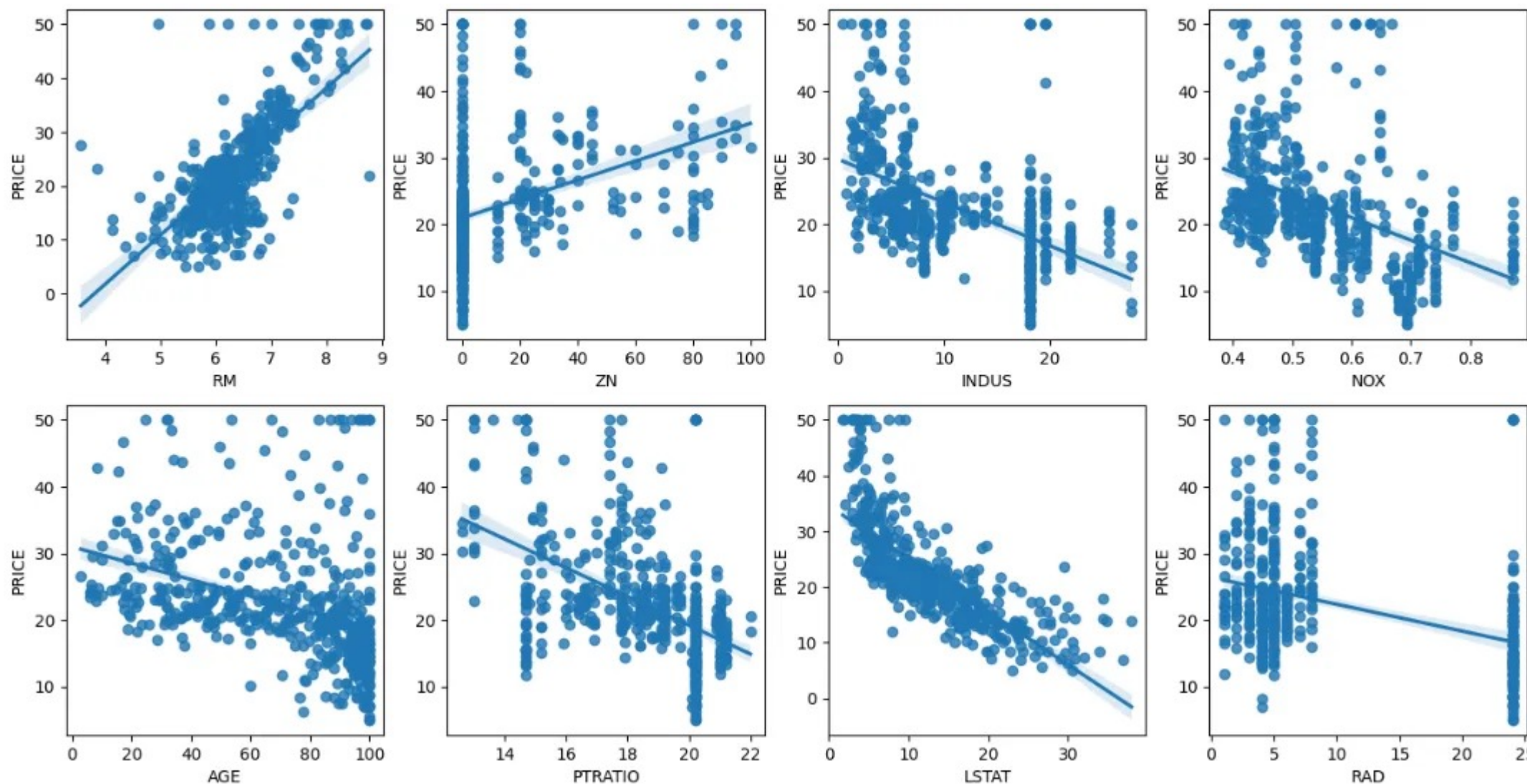
- CRIM: 지역별 범죄 발생률
- ZN: 25,000평방피트를 초과하는 거주 지역의 비율
- INDUS: 비상업 지역 넓이 비율
- CHAS: 찰스강에 대한 더미 변수(강의 경계에 위치한 경우는 1, 아니면 0)
- NOX: 일산화질소 농도
- RM: 거주할 수 있는 방 개수
- AGE: 1940년 이전에 건축된 소유 주택의 비율
- DIS: 5개 주요 고용센터까지의 가중 거리
- RAD: 고속도로 접근 용이도
- TAX: 10,000달러당 재산세율
- PTRATIO: 지역의 교사와 학생 수 비율
- B: 지역의 흑인 거주 비율
- LSTAT: 하위 계층의 비율
- MEDV: 본인 소유의 주택 가격(중앙값)

#5.4.3. LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

```
[ ] # 2개의 행과 4개의 열을 가진 subplots를 이용. axs는 4*2개의 ax를 가짐
fig, axs = plt.subplots(figsize=(16, 8), ncols=4, nrow=2)
lm_features = ['RM', 'ZN', 'INDUS', 'NOX', 'AGE', 'PTRATIO', 'LSTAT', 'RAD']
for i, feature in enumerate(lm_features):
    row = int(i/4)
    col = i%4
    # 시본의 regplot을 이용해 산점도와 선형 회귀 직선을 함께 표현
    sns.regplot(x=feature, y='PRICE', data=bostonDF, ax=axs[row][col])
```

- RM과 LSTAT의 PRICE 영향도가 가장 두드러짐

- RM(방 개수) : 양 방향의 선형성
⇒ 방의 크기가 클수록 가격 증가
- LSTAT(하위 계층의 비율) : 음 방향의 선형성
⇒ LSTAT이 적을 수록 PRICE가 증가



#5.4.3. LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_data, y_target, test_size=0.3, random_state=156)

# 선형 회귀 OLS로 학습/예측/평가 수행
lr = LinearRegression()
lr.fit(X_train, y_train)
y_preds = lr.predict(X_test)
mse = mean_squared_error(y_test, y_preds)
rmse = np.sqrt(mse)

print('MSE: {0:.3f}, RMSE: {1:.3F}'.format(mse, rmse))
print('Variance score: {0:.3f}'.format(r2_score(y_test, y_preds)))
```

```
⇒ MSE: 17.297, RMSE: 4.159
Variance score: 0.757
```

```
[ ] print('절편 값:', lr.intercept_)
    print('회귀 계수값:', np.round(lr.coef_, 1))
```

```
⇒ 절편 값: 40.995595172164826
회귀 계수값: [ -0.1   0.1   0.    3.   -19.8   3.4   0.   -1.7   0.4  -0.   -0.9   0.
 -0.6]
```

#5.4.3. LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

```
# 회귀 계수를 큰 값 순으로 정렬하기 위해 Series로 생성. 인덱스 칼럼명에 유의
coeff = pd.Series(data=np.round(lr.coef_, 1), index=X_data.columns)
coeff.sort_values(ascending=False)
```

	0
RM	3.4
CHAS	3.0
RAD	0.4
ZN	0.1
INDUS	0.0
AGE	0.0
TAX	-0.0
B	0.0
CRIM	-0.1
LSTAT	-0.6
PTRATIO	-0.9
DIS	-1.7
NOX	-19.8

dtype: float64

- RM이 양의 값으로 회귀 계수가 가장 큼, NOX 피처의 회귀 계수 -값이 너무 큼.
- 최적화를 수행하며 피처 coefficients의 변화 보기

```
[ ] from sklearn.model_selection import cross_val_score

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)

lr = LinearRegression()

# cross_val_score()로 5 폴드 세트의 MSE를 구한 뒤 이를 기반으로 다시 RMSE 구함
neg_mse_scores = cross_val_score(lr, X_data, y_target, scoring="neg_mean_squared_error", cv=5)
rmse_scores = np.sqrt(-1*neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

# cross_val_score(scoring="neg_mean_squared_error")로 반환된 값은 모두 음수임
print(' 5 folds 의 개별 Negative MSE scores: ', np.round(neg_mse_scores, 2))
print(' 5 folds 의 개별 RMSE scores: ', np.round(rmse_scores, 2))
print(' 5 folds 의 평균 RMSE: {0:.3f}'.format(avg_rmse))
```

```
5 folds 의 개별 Negative MSE scores: [-12.46 -26.05 -33.07 -80.76 -33.31]
5 folds 의 개별 RMSE scores: [3.53 5.1 5.75 8.99 5.77]
5 folds 의 평균 RMSE: 5.829
```

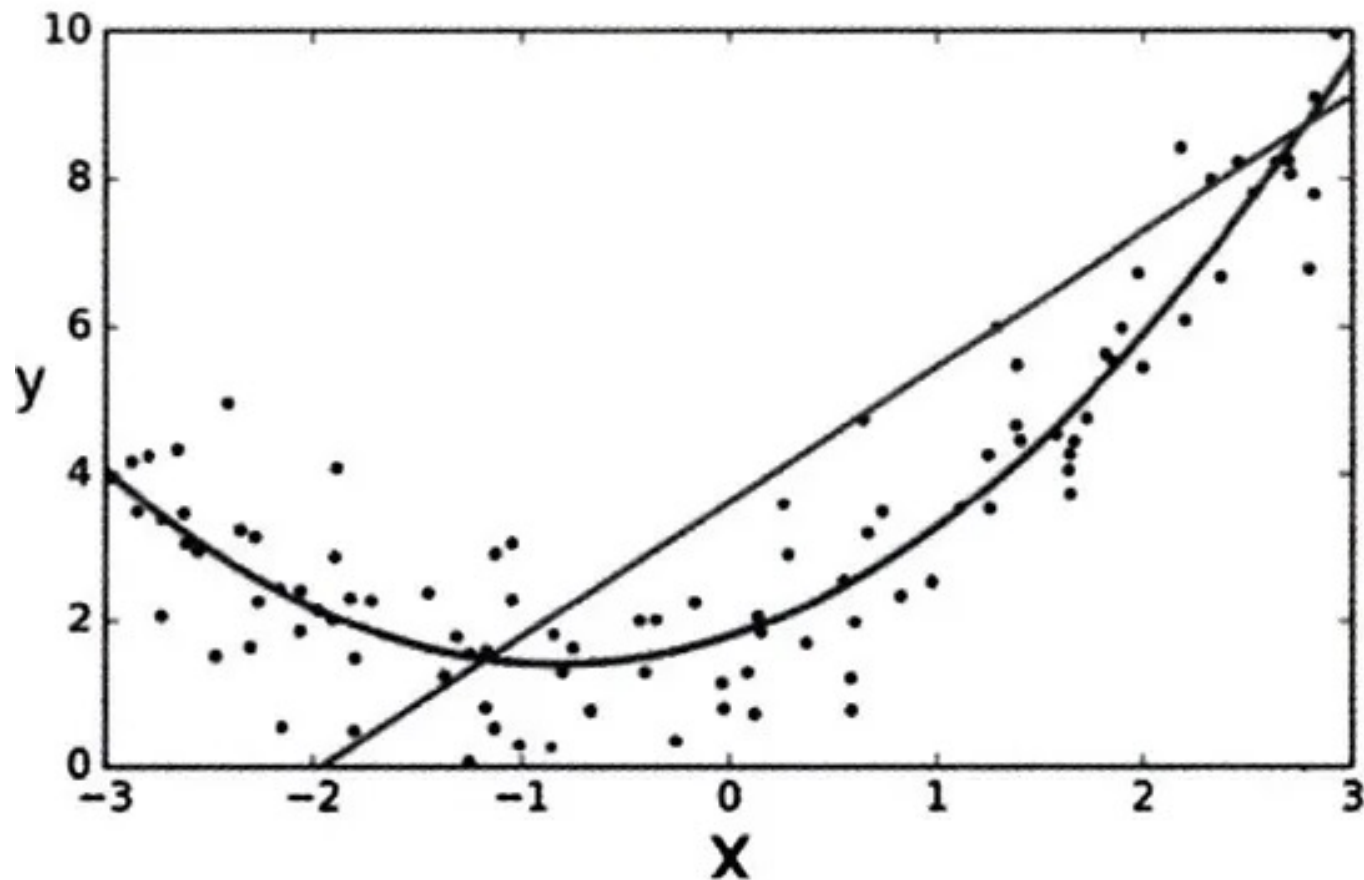
5.5 다항회귀



#5.5.1. 다항 회귀 이해

다항 회귀

- 회귀가 독립변수의 단항식이 아닌 2차, 3차 방정식과 같은 다항식으로 표현
- 주의할 점 !! $\Rightarrow$ 다항회귀는 선형 회귀(비선형 회귀X)
 - 선형/비선형 회귀를 나누는 기준 : 회귀 계수가 선형/비선형
- 피쳐 X와 Target Y값의 관계를 다항 회귀 곡선형으로 표현하는 것이 더 예측 성능이 높음



〈 주어진 데이터 세트에서 다항 회귀가 더 효과적임 〉

#5.5.1. 다항 회귀 이해

- 사이킷런은 다항 회귀를 위한 클래스를 명시적으로 제공 X
- 다항 회귀는 선형 회귀이기에 비선형 함수를 선형 모델에 적용시키는 방법을 사용하여 구현
 - ⇒ PolynomialFeatures 클래스
 - 클래스를 통해 피처를 Polynomial(다항식)피처로 변환
 - degree 파라미터를 통해 입력 받은 단항식 피처를 degree에 해당하는 다항식 피처로 변환

#5.5.1. 다항 회귀 이해

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np
# 다항식으로 변환한 단항식 생성, [[0,1],[2,3]]의 2X2 행렬 생성
X = np.arange(4).reshape(2,2)
print('일차 단항식 계수 feature:\n', X)

# degree = 2 인 2차다항식으로 변환하기 위해 PolynomialFeatures를 이용해 변환
poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print('변환된 2차 다항식 계수 feature:\n', poly_ftr)
```

```
일차 단항식 계수 feature:
[[0 1]
 [2 3]]
변환된 2차 다항식 계수 feature:
[[1. 0. 1. 0. 0. 1.]
 [1. 2. 3. 4. 6. 9.]]
```

단항 계수 피쳐 $[x_1, x_2]$



$[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

첫번째 입력 단항 계수 피쳐 $[x_1 = 0, x_2 = 1]$



$[1, x_1 = 0, x_2 = 1, x_1^2 = 0, x_1x_2 = 0, x_2^2 = 0]$

두번째 입력 단항 계수 피쳐 $[x_1 = 2, x_2 = 3]$



$[1, 2, 3, 4, 6, 9]$

#5.5.1. 다항 회귀 이해

```
[ ] def polynomial_fuc(X):  
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3  
    return y  
  
X = np.arange(4).reshape(2,2)  
print('일차 다항식 계수 feature:\n', X)  
y = polynomial_fuc(X)  
print('삼차 다항식 결정값: \n', y)
```

```
⇒ 일차 다항식 계수 feature:  
[[0 1]  
 [2 3]]  
삼차 다항식 결정값:  
[ 5 125]
```

- 3차 다항 회귀 함수 임의로 설정, 회귀 계수 예측
- 3차 다항 회귀 결정 함수식 : $y = 1 + 2x_1 + 3x_1^2 + 4x_2^3$
- 이를 위한 함수 polynomial_func() 생성
 - 3차 다항 계수 피쳐 값이 입력되면 결정 값 반환

```
# 3차 다항식 변환  
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)  
print('3차 다항식 계수 feature:\n', poly_ftr)  
  
# Linear Regression에 3차 다항식 계수 feature와 3차 다항식 결정값으로 학습 후 회귀 계수 확인  
model = LinearRegression()  
model.fit(poly_ftr, y)  
print('Polynomial 회귀 계수\n', np.round(model.coef_, 2))  
print('Polynomial 회귀 Shape: ', model.coef_.shape)
```

```
⇒ 3차 다항식 계수 feature:  
[[ 1.  0.  1.  0.  0.  1.  0.  0.  0.  1.]  
 [ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]]  
Polynomial 회귀 계수  
[0.  0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]  
Polynomial 회귀 Shape: (10,)
```

단항 계수 피쳐 $[x_1, x_2]$ 를

3차 다항 계수

$[1, x_1, x_2, x_1^2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_1^3]$

과 같이 10개의 다항 계수로 변환

사이킷런의 PolynomialFeatures로

피쳐 변환 후

LinearRegression 클래스로

다항회귀 구현

#5.5.1. 다항 회귀 이해

- 사이킷런의 Pipeline객체 이용하여 한번에 다항 회귀 구현

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
import numpy as np

def polynomial_func(X):
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    return y

# Pipeline 객체로 Streamline 하게 Polynomial Feature 변환과 Linear Regression을 연결
model = Pipeline([('poly', PolynomialFeatures(degree=3)), ('linear', LinearRegression())])

X = np.arange(4).reshape(2,2)
y = polynomial_func(X)

model = model.fit(X, y)
print('Polynomial 회귀 계수\\n', np.round(model.named_steps['linear'].coef_, 2))
```

Polynomial 회귀 계수

[0. 0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]

#5.5.2. 다항 회귀를 이용한 과소적합 및 과적합 이해

- 다항식의 차수가 높아질수록 매우 복잡한 피쳐 간의 관계까지 모델링 가능
- But, 차수가 높아질수록 과적합의 문제가 크게 발생!
- 다항 회귀를 이용해 과소적합, 과적합 문제를 잘 보여주는 예제

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
%matplotlib inline

#임의의 값으로 구성된 X값에 대해 코사인 변환 값을 반환
def true_fun(X):
    return np.cos(1.5*np.pi*X)

#X는 0부터 1까지 30개의 임의의 값을 순서대로 샘플링한 데이터 입니다.
np.random.seed(0)
n_samples = 30
X = np.sort(np.random.rand(n_samples))

#y값은 코사인 기반의 true_fun()에서 약간의 노이즈 변동 값을 더한 값입니다.
y = true_fun(X) + np.random.randn(n_samples)*0.1
```

```
plt.figure(figsize=(14,5))
degrees = [1,4,15]

#다항 회귀의 차수(degree)를 1, 4, 15로 각각 변화시키면서 비교합니다.
for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i+1)
    plt.setp(ax, xticks=(), yticks=())

    #개별 degree별로 Polynomial 변환합니다.
    polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
    linear_regression = LinearRegression()
    pipeline = Pipeline([("polynomial_features", polynomial_features), ("linear_regression", linear_regression)])
    pipeline.fit(X.reshape(-1,1),y)

    #교차 검증으로 다항 회귀 평가
    scores = cross_val_score(pipeline, X.reshape(-1,1), y, scoring="neg_mean_squared_error", cv=10)
    #Pipeline을 구성하는 세부 객체를 접근하는 named_steps['객체명']을 이용해 회귀계수 추출
    coefficients = pipeline.named_steps['linear_regression'].coef_
    print('\nDegree {0} 회귀 계수는 {1}입니다.'.format(degrees[i], np.round(coefficients,2)))
    print('Degree {0} MSE는 {1} 입니다.'.format(degrees[i], -1*np.mean(scores)))

    #0부터 1까지 테스트 데이터 세트를 100개로 나눠 예측을 수행
    #테스트 데이터 세트에 회귀 예측을 수행하고 예측 곡선과 실제 곡선을 그려서 비교
    X_test = np.linspace(0,1,100)
    #예측값 곡선
    plt.plot(X_test, pipeline.predict(X_test[:,np.newaxis]), label='Model')
    #실제값 곡선
    plt.plot(X_test, true_fun(X_test), '--', label='True Function')
    plt.scatter(X, y, edgecolor='b', s=20, label='Samples')

    plt.xlabel("x"); plt.ylabel("y"); plt.xlim((0,1)); plt.ylim((-2,2)); plt.legend(loc="best")
    plt.title("Degree {} MSE = {:.2e} (+/- {:.2e})".format(degrees[i], -scores.mean(), scores.std()))

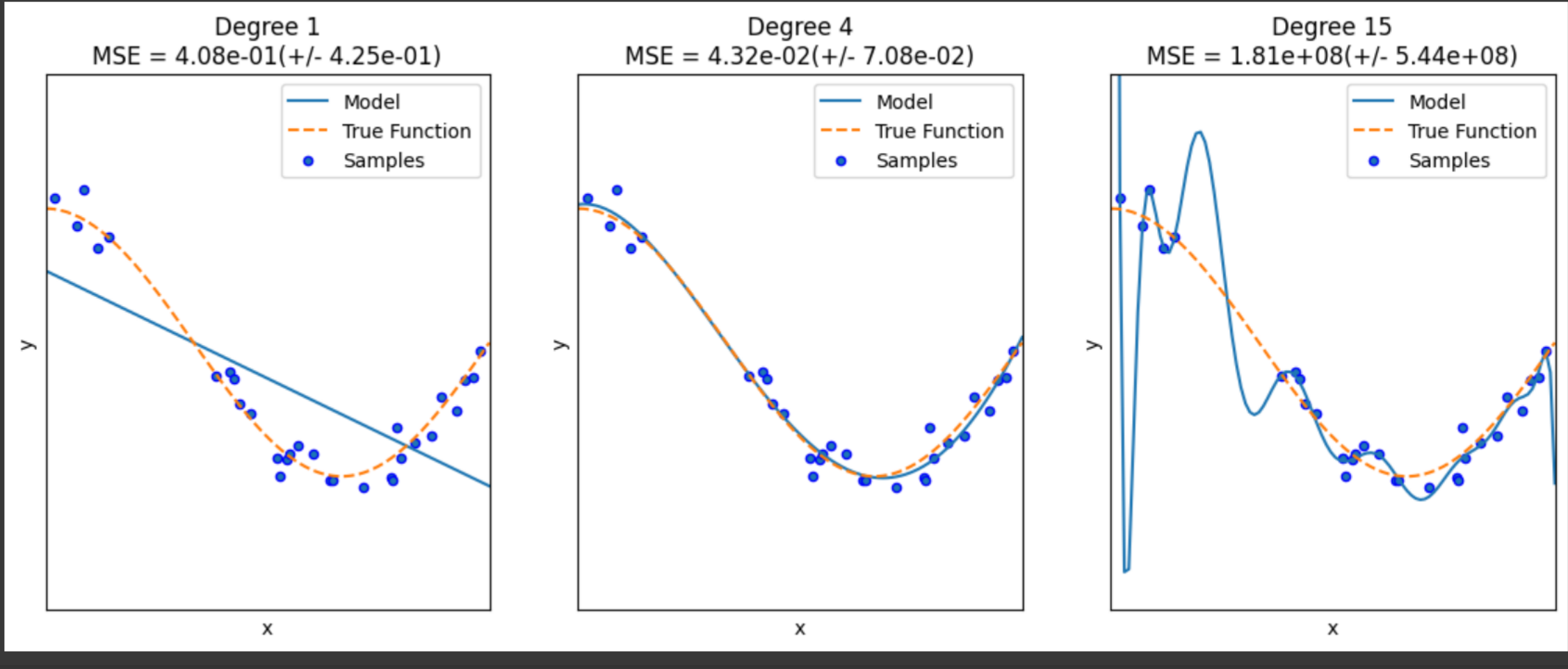
plt.show()
```

#5.5.2. 다항 회귀를 이용한 과소적합 및 과적합 이해

Degree 1 회귀 계수는 [-1.61]입니다.
Degree 1 MSE는 0.4077289625098685 입니다.

Degree 4 회귀 계수는 [0.47 -17.79 23.59 -7.26]입니다.
Degree 4 MSE는 0.04320874987232064 입니다.

Degree 15 회귀 계수는 [-2.98293000e+03 1.03899390e+05 -1.87416123e+06 2.03716219e+07
-1.44873283e+08 7.09315363e+08 -2.47065792e+09 6.24561050e+09
-1.15676510e+10 1.56894936e+10 -1.54006023e+10 1.06457264e+10
-4.91377530e+09 1.35919645e+09 -1.70380786e+08]입니다.
Degree 15 MSE는 181238256.56423894 입니다.



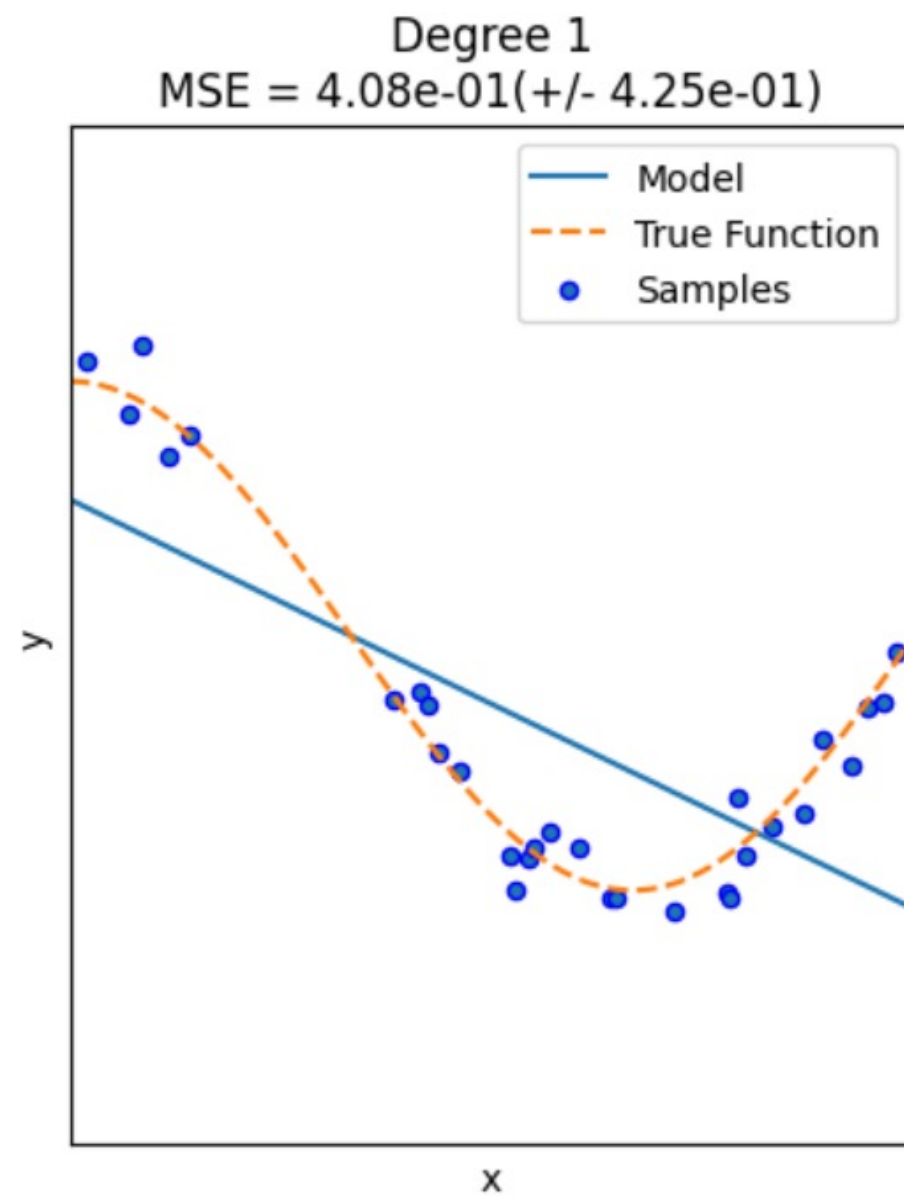
- 실선 : 다항 회귀 예측 곡선
- 점선 : 실제 데이터 세트 X, Y 코사인 곡선
- 학습 데이터는 0-1까지의 30개 임의의 X값과 그에 따른 코사인 Y값에 잡음을 변동 값으로 추가해 구성했으며 MSE 평가는 학습 데이터를 10개의 교차 검증 세트로 나눠 측정해서 평균한 것

좋은 예측 모델 :
학습 데이터의 패턴을
잘 반영하면서도 복잡하지 않은
균형 잡힌(Balanced) 모델

#5.5.3. 편향-분산 트레이드오프(Bias-Variance Trade off)

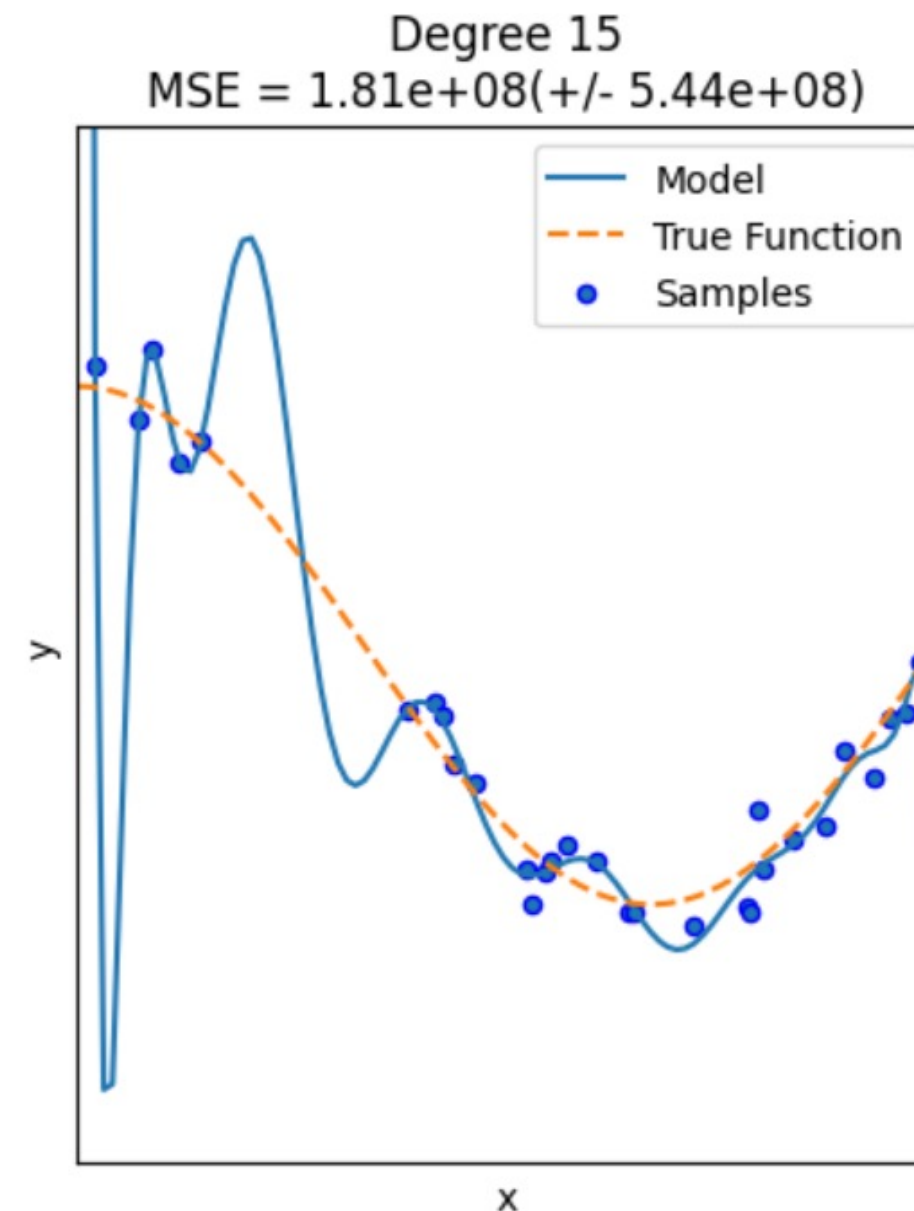
고편향(High Bias)성

- 지나치게 한 방향으로 치우친 경향
e.g) Degree1과 같은 매우 단순한 모델



고분산(High Variance)성

- 지나치게 높은 변동성
e.g) Degree15와 같은 모델은 학습 데이터 하나하나의 특성을 반영한 매우 복잡한 모델



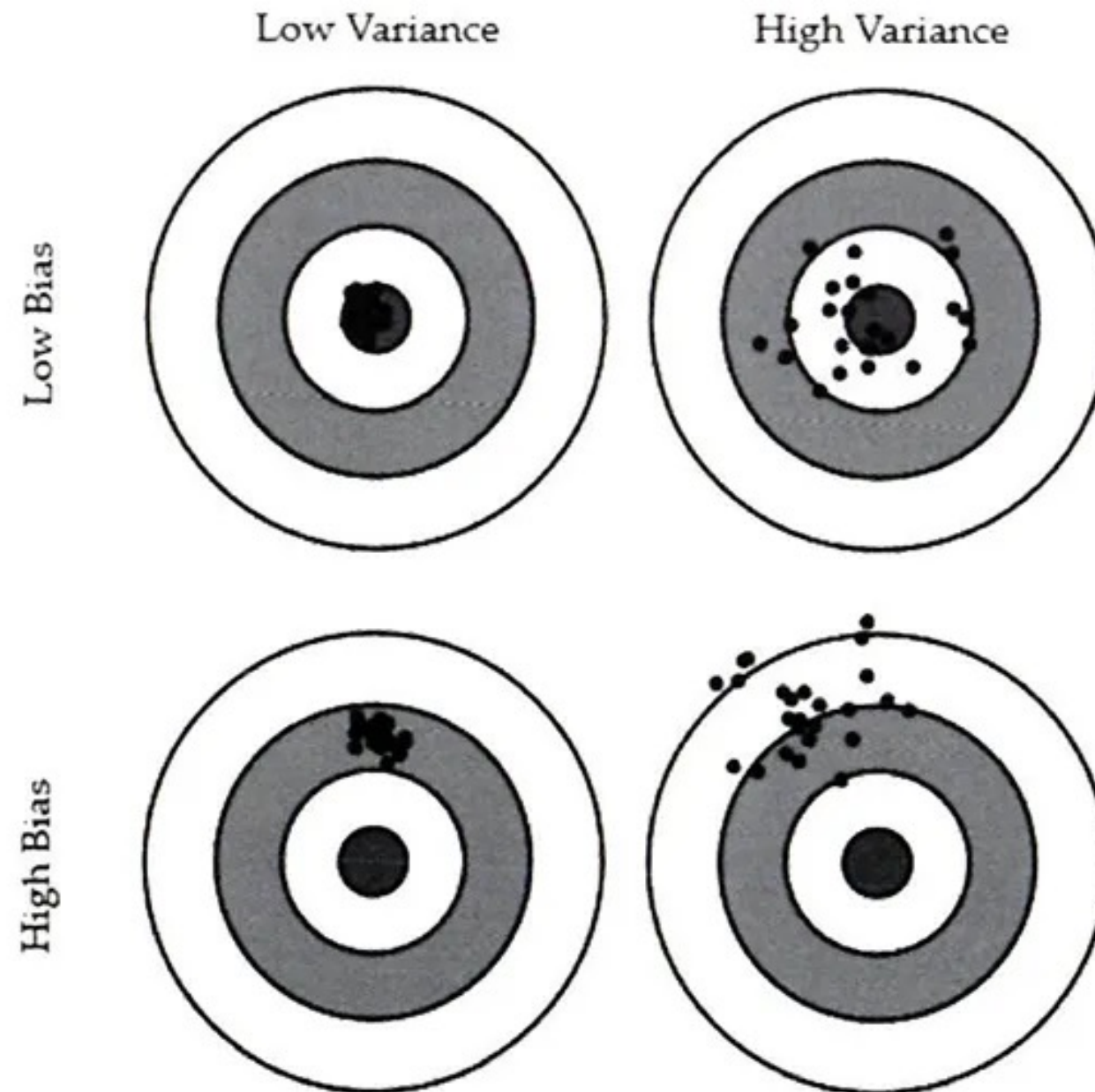
#5.5.3. 편향-분산 트레이드오프(Bias-Variance Trade off)

저편향/저분산

- 예측 결과가 실제 결과에 매우 근접
- 특정 부분에 집중
- 아주 뛰어난 성능

고편향/저분산

- 정확한 결과에서 벗어남
- 예측이 특정 부분에 집중



〈 편향과 분산의 고/저에 따른 표현. 〉

<http://scott.fortmann-roe.com/docs/BiasVariance.html>에서 발췌 >

저편향/고분산

- 예측 결과가 실제 결과에 비교적 근접
- 예측 결과가 실제 결과를 중심으로 꽤 넓은 부분에 분포

고편향/고분산

- 정확한 예측 결과를 벗어남
- 넓은 부분에 분포

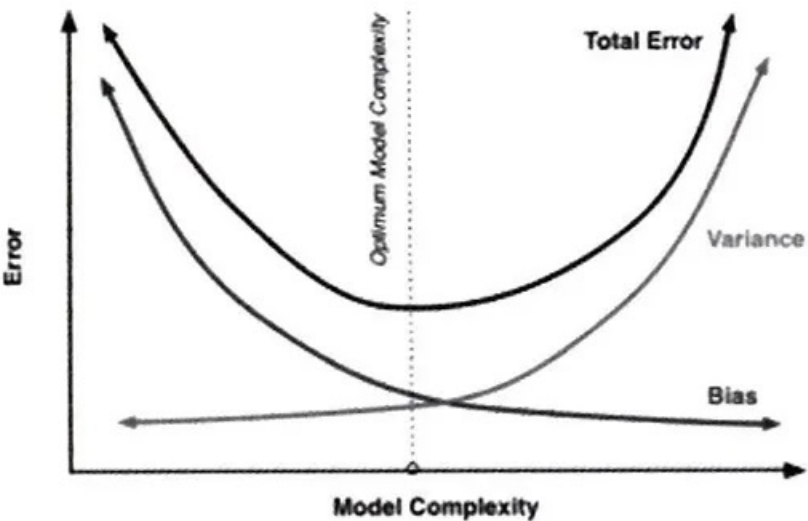
#5.5.3. 편향-분산 트레이드오프(Bias-Variance Trade off)

과소적합

편향 $\uparrow$ $\longrightarrow$ 분산 $\downarrow$

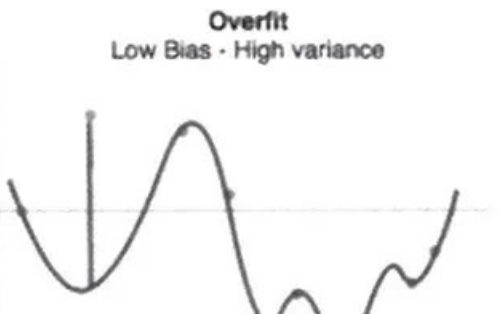
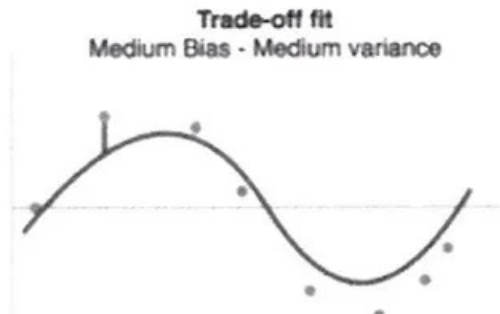
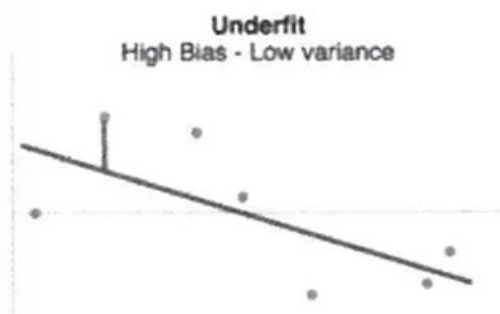
과적합

분산 $\uparrow$ $\longrightarrow$ 편향 $\downarrow$



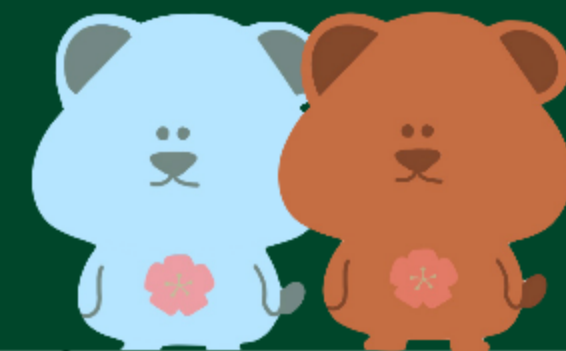
‘골디락스 지점’

편향을 낮추고 분산을 높이면서
전체 오류가 가장 낮아지는 지점



< 편향과 분산에 따른 전체 오류 값(Total Error) 곡선. <http://scott.fortmann-roe.com/docs/BiasVariance.html>에서 발췌. >

편향 분산이 서로 트레이드오프를 이루며
오류 Cost값이 최대한 낮아지는 모델 구축하자!



5.6 규제 선행 회귀 - 릿지, 라쏘, 엘라스틱넷 ~ 5.8 회귀 트리

5.6 규제 선행 회귀

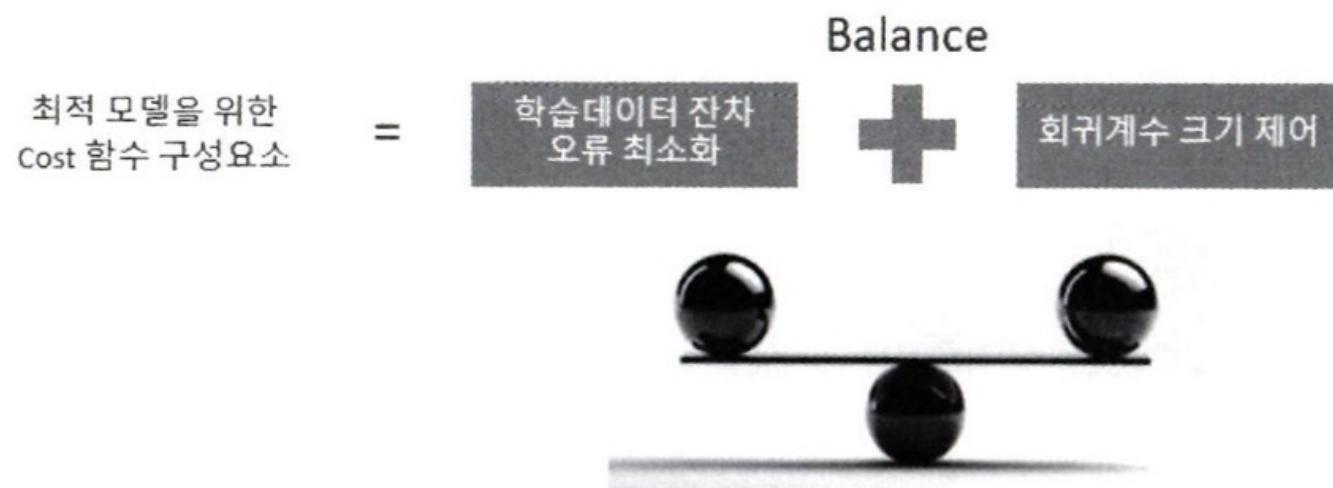


#5.6.1 규제 선형 모델의 개요

이전의 선형 모델 비용함수 : RSS 최소화만 고려

>> 변동성이 심해지고, 테스트 데이터 예측 성능 저하

New!



비용 함수 목표 = $\text{Min}(\text{RSS}(W) + \alpha * \|W\|_2^2)$

*Alpha 값을 크게 >> W값을 작게 >> 과적합 개선

Alpha 값을 작게 >> W값 커져도 상쇄가능

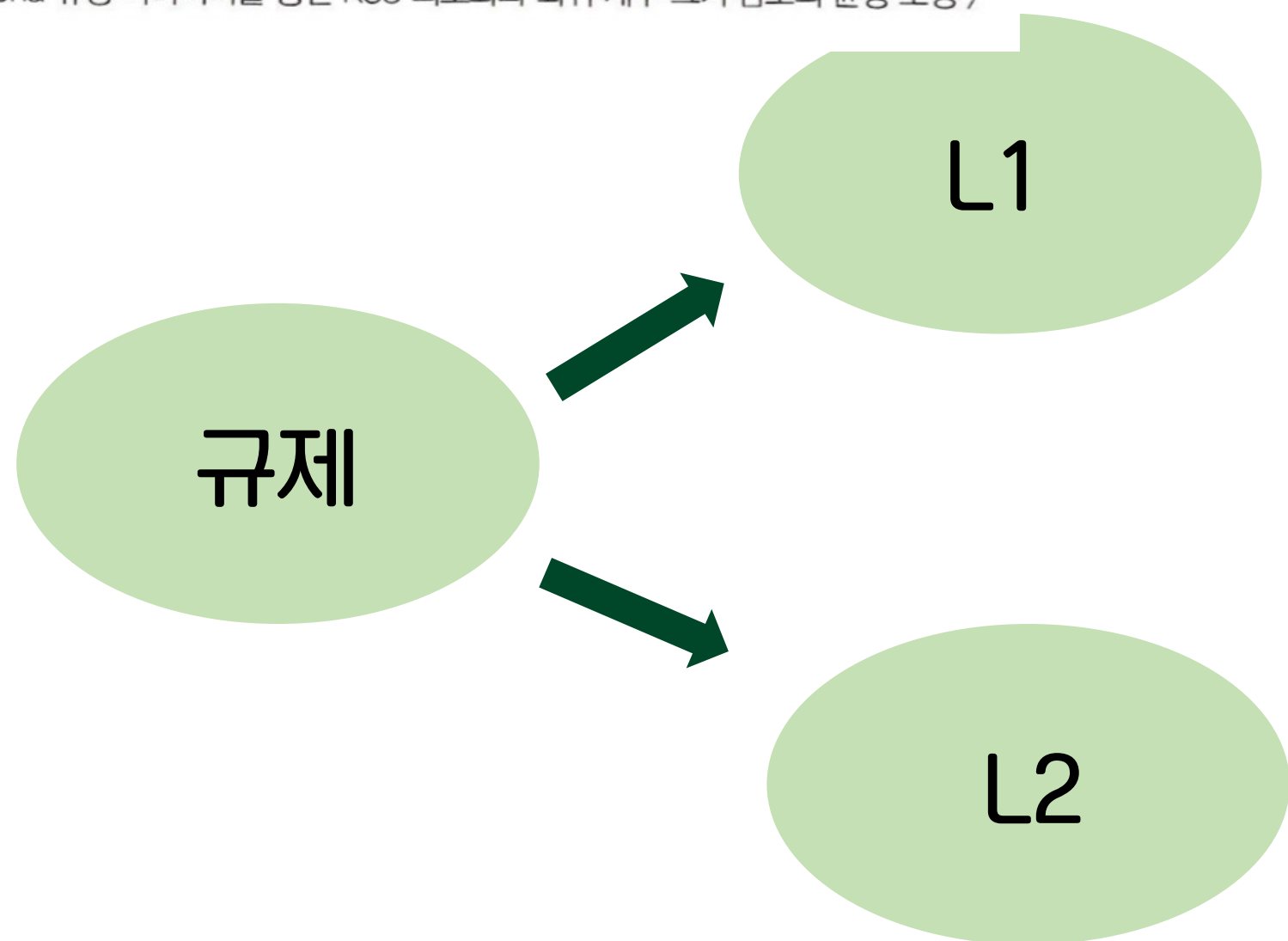
#5.6.1 규제 선형 모델의 개요



RSS(W) 최소화

회귀 계수 W 감소

(alpha 튜닝 파라미터를 통한 RSS 최소화과 회귀 계수 크기 감소의 균형 조정)



라쏘 회귀

W 절댓값에 패널티 부여

릿지 회귀

W 제곱에 패널티 부여, 영향력이 크지 않은

회귀 계수에 0을 반환

#5.6.2 릿지 회귀

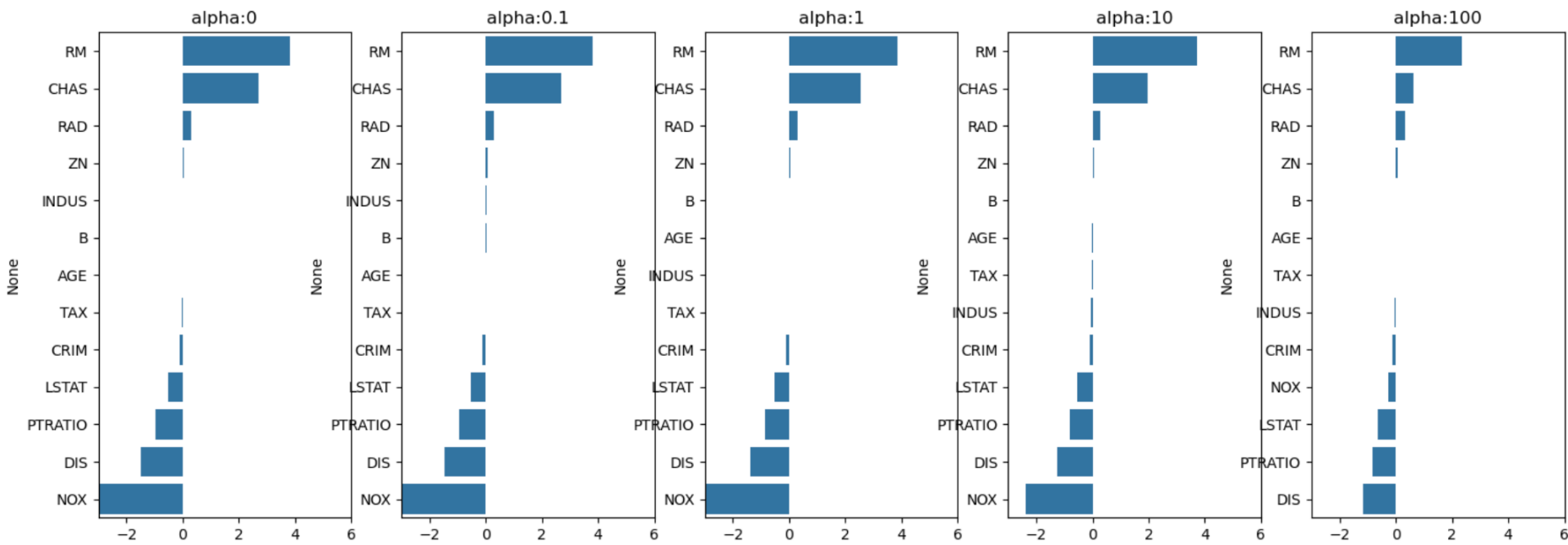
릿지 회귀 적용 : Ridge 클래스의 alpha 파라미터 > L2 규제 계수에 해당

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

# alpha=10으로 설정해 릿지 회귀 수행
# RMSE 평균제곱오차제곱근
ridge=Ridge(alpha=10)
neg_mse_scores=cross_val_score(ridge,X_data,y_target,scoring="neg_mean_squared_error",cv=5)
rmse_scores=np.sqrt(-1*neg_mse_scores)
avg_rmse=np.mean(rmse_scores)

print('5 folds의 개별 Negative MSE score :',np.round(neg_mse_scores,3))
print('5 folds의 개별 RMSE scores :',np.round(rmse_scores,3))
print('5 folds의 평균 RMSE :{0:.3f}'.format(avg_rmse))
```

5 folds의 개별 Negative MSE score : [-11.422 -24.294 -28.144 -74.599 -28.517]
5 folds의 개별 RMSE scores : [3.38 4.929 5.305 8.637 5.34]
5 folds의 평균 RMSE :5.518

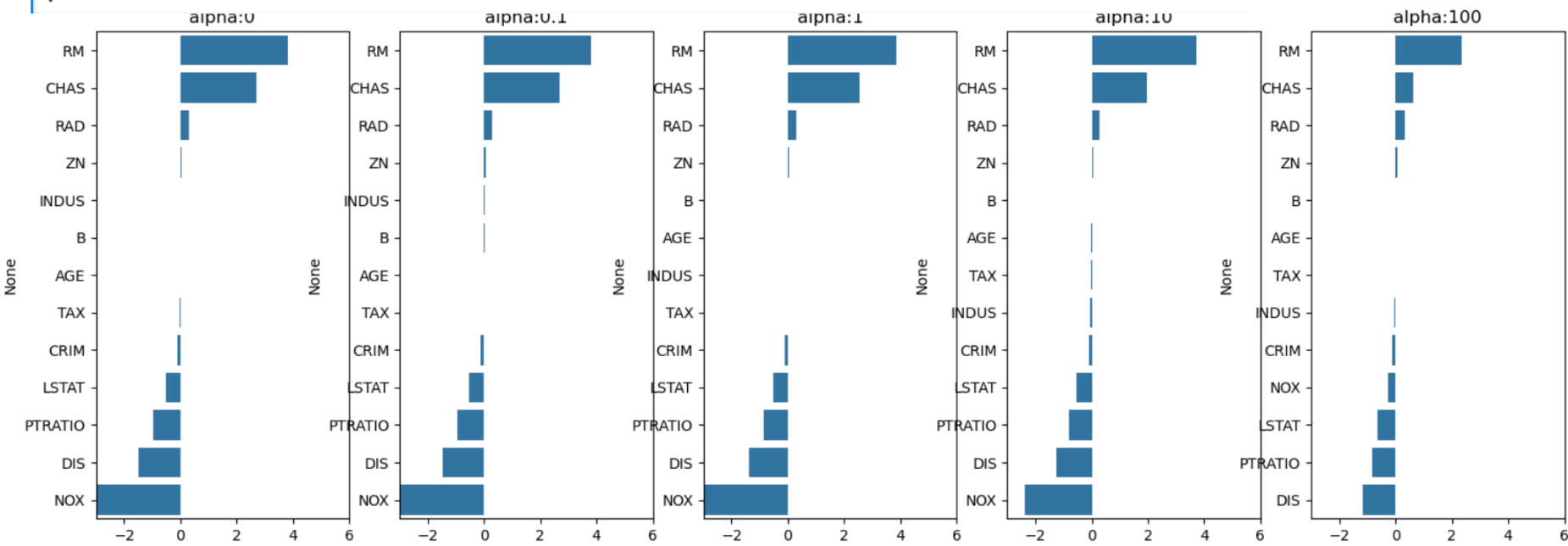


	alpha:0	alpha:0.1	alpha:1	alpha:10	alpha:100
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

#5.6.2 릿지 회귀

```
# 각 alpha에 따른 회귀 계수 값을 시각화하기 위해 5개의 열로 된 맷플롯립 축 생성
fig,axs=plt.subplots(figsize=(18,6),nrows=1,ncols=5)
# 각 alpha에 따른 회귀 계수 값을 데이터로 저장
coeff_df=pd.DataFrame()

#alpha 리스트 값을 입력해 회귀 계수 값 시각화 및 데이터 저장, pos는 axis의 위치 저장
for pos, alpha in enumerate(alphas):
    ridge=Ridge(alpha=alpha)
    ridge.fit(X_data,y_target)
    # alpha에 따른 피러별로 회귀 계수를 Series로 변환하고 칼럼으로 추가
    coeff=pd.Series(data=ridge.coef_,index=X_data.columns)
    colname='alpha:'+str(alpha)
    coeff_df[colname]=coeff
    # 막대 그래프로 각 alpha 값에서의 회귀 계수를 시각화, 높은 순으로 표현
    coeff=coeff.sort_values(ascending=False)
    axs[pos].set_title(colname)
    axs[pos].set_xlim(-3,6)
    sns.barplot(x=coeff.values,y=coeff.index,ax=axs[pos])
plt.show()
```



```
# alpha 값 증가에 따른 회귀 계수 변화
ridge_alphas=[0,0.1,1,10,100]
sort_column='alpha:'+str(alphas[0])
coeff_df.sort_values(by=sort_column,ascending=False)
```

	alpha:0	alpha:0.1	alpha:1	alpha:10	alpha:100
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

>> alpha 값이 증가함에 따라 회귀 계수가 작아진다 but 0으로는 안 만든다

#5.6.3 라쏘 회귀

라쏘 회귀 적용 : Ridge 클래스의 alpha 파라미터 > L1 규제 계수에 해당

회귀 계수를 급격하게 감소시켜 0으로 만들고 제거

<> 릿지 회귀 : 회귀 계수의 크기를 감소시키는데 0으로는 안 만든다

```
from sklearn.linear_model import Lasso, ElasticNet

#alpha값에 따른 회귀 모델의 평균 RMSE를 출력, 회귀 계수는 df로 변환
def get_linear_reg_eval(model_name, params=None, X_data_n=None, y_target_n=None, verbose=True):
    coeff_df = pd.DataFrame()
    if verbose : print('##### ', model_name , '#####')
    for param in params:
        if model_name == 'Ridge': model = Ridge(alpha=param)
        elif model_name == 'Lasso': model = Lasso(alpha=param)
        elif model_name == 'ElasticNet': model = ElasticNet(alpha=param, l1_ratio=0.7)
        neg_mse_scores = cross_val_score(model, X_data_n,
                                         y_target_n, scoring="neg_mean_squared_error", cv = 5)
        avg_rmse = np.mean(np.sqrt(-1 * neg_mse_scores))
        print('alpha {0}일 때 5 폴드 세트의 평균 RMSE: {1:.3f} '.format(param, avg_rmse))
        # cross_val_score는 evaluation metric만 반환하므로 모델을 다시 학습하여 회귀 계수 추출
        model.fit(X_data , y_target)
        # alpha에 따른 피쳐별 회귀 계수를 Series로 변환하고 이를 DataFrame의 컬럼으로 추가.
        coeff = pd.Series(data=model.coef_ , index=X_data.columns )
        colname='alpha:'+str(param)
        coeff_df[colname] = coeff
    return coeff_df
```

#5.6.3 라쏘 회귀

alpha 값 변화에 따른 회귀 계수 출력

```
lasso_alphas=[0.07,0.1,0.5,1,3]
coeff_lasso_df=get_linear_reg_eval('Lasso',params=lasso_alphas,X_data_n=X_data, y_target_n=y_target)
```

Lasso #####
alpha 0.07일 때 5 폴드 세트의 평균 RMSE: 5.612
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.615
alpha 0.5일 때 5 폴드 세트의 평균 RMSE: 5.669
alpha 1일 때 5 폴드 세트의 평균 RMSE: 5.776
alpha 3일 때 5 폴드 세트의 평균 RMSE: 6.189

>> alpha 값이 0.07일 때가 best
Ridge 평균보다는 떨어지지만,
LinearRegression 평균보다는 항상

#반환된 coeff_lasso_df를 첫 번째 칼럼순으로 내림차순 정렬해 회귀계수 df 출력
sort_column='alpha:'+str(lasso_alphas[0])
coeff_lasso_df.sort_values(by=sort_column,ascending=False)

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.789725	3.703202	2.498212	0.949811	0.000000
CHAS	1.434343	0.955190	0.000000	0.000000	0.000000
RAD	0.270936	0.274707	0.277451	0.264206	0.061864
ZN	0.049059	0.049211	0.049544	0.049165	0.037231
B	0.010248	0.010249	0.009469	0.008247	0.006510
NOX	-0.000000	-0.000000	-0.000000	-0.000000	0.000000
AGE	-0.011706	-0.010037	0.003604	0.020910	0.042495
TAX	-0.014290	-0.014570	-0.015442	-0.015212	-0.008602
INDUS	-0.042120	-0.036619	-0.005253	-0.000000	-0.000000
CRIM	-0.098193	-0.097894	-0.083289	-0.063437	-0.000000
LSTAT	-0.560431	-0.568769	-0.656290	-0.761115	-0.807679
PTRATIO	-0.765107	-0.770654	-0.758752	-0.722966	-0.265072
DIS	-1.176583	-1.160538	-0.936605	-0.668790	-0.000000

>> 일부 피처는 alpha 값이 증가함에 따라 회귀 계수가 0

#5.6.4 엘라스틱넷 회귀

엘라스틱넷 회귀 : L2와 L1 규제를 결합한 회귀

$RSS(W) + \alpha_2 * \|W\|_2^2 + \alpha_1 * \|W\|_1$ >> 이 식을 최소화하는 W 찾기

L2 규제 추가

>> 라쏘 회귀의 중요 피처만 선택&다른 피처의 w를 모두 0으로 만드는 성향 완화

```
ElasticNet(alpha=param, l1_ratio=0.7)
```

주요 파라미터

1. Alpha = a(L1 규제의 alpha값)+b(L2 규제의 alpha값)
2. L1_ratio=a/(a+b)

#5.6.4 엘라스틱넷 회귀

```
# 엘라스틱넷에 사용될 alpha 파라미터의 값들을 정의, l1_ratio는 0.7로 고정
elastic_alphas=[0.07,0.1,0.5,1,3]
coeff_elastic_df=get_linear_reg_eval('ElasticNet',params=lasso_alphas,X_data_n=X_data, y_target_n=y_target)

##### ElasticNet #####
alpha 0.07일 때 5 폴드 세트의 평균 RMSE: 5.542
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.526
alpha 0.5일 때 5 폴드 세트의 평균 RMSE: 5.467
alpha 1일 때 5 폴드 세트의 평균 RMSE: 5.597
alpha 3일 때 5 폴드 세트의 평균 RMSE: 6.068
```

>> alpha 값이 0.05일 때가 best

```
# 반환된 coeff_elastic_df를 첫 번째 칼럼순으로 내림차순 정렬해 회귀계수 df 출력
sort_column='alpha:'+str(elastic_alphas[0])
coeff_elastic_df.sort_values(by=sort_column,ascending=False)
```

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.574162	3.414154	1.918419	0.938789	0.000000
CHAS	1.330724	0.979706	0.000000	0.000000	0.000000
RAD	0.278880	0.283443	0.300761	0.289299	0.146846
ZN	0.050107	0.050617	0.052878	0.052136	0.038268
B	0.010122	0.010067	0.009114	0.008320	0.007020
AGE	-0.010116	-0.008276	0.007760	0.020348	0.043446
TAX	-0.014522	-0.014814	-0.016046	-0.016218	-0.011417
INDUS	-0.044855	-0.042719	-0.023252	-0.000000	-0.000000
CRIM	-0.099468	-0.099213	-0.089070	-0.073577	-0.019058
NOX	-0.175072	-0.000000	-0.000000	-0.000000	-0.000000
LSTAT	-0.574822	-0.587702	-0.693861	-0.760457	-0.800368
PTRATIO	-0.779498	-0.784725	-0.790969	-0.738672	-0.423065
DIS	-1.189438	-1.173647	-0.975902	-0.725174	-0.031208

>> 라쏘보다 상대적으로 0되는 값 적음

#5.6.5 선형 회귀 모델을 위한 데이터 변환

* 중요 피처들이나 타깃값의 분포도가 심하게 왜곡되었을 때만 변환작업

1. 사이킷런으로 피처 데이터 세트에 적용하는 방법

1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

2. 타깃값 > 로그변환 적용

#5.6.5 선형 회귀 모델을 위한 데이터 변환

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PolynomialFeatures
#method는 표준 정규 변환(Standard), 최댓값/최솟값 정규화(MinMax), 로그 변환(Log) 결정
#p_degree는 다항식 특성을 추가할 때 적용, 2이상 부여안함

def get_scaled_data(method='None', p_degree=None, input_data=None):
    if method=='Standard':
        scaled_data=StandardScaler().fit_transform(input_data)
    elif method=='MinMax':
        scaled_data=MinMaxScaler().fit_transform(input_data)
    elif method=='Log':
        scaled_data=np.log1p(input_data)
    else:
        scaled_data=input_data
    if p_degree!=None:
        scaled_data=PolynomialFeatures(degree=p_degree,include_bias=False).fit_transform(scaled_data)
    return scaled_data
```



```
# Ridge의 alpha값을 다르게 적용하고 다양한 데이터 변환방법에 따른 RMSE 추출.
alphas = [0.1, 1, 10, 100]

#변환 방법은 모두 5개, 원본 그대로, 표준정규분포, 표준정규분포+다항식 특성
# 최대/최소 정규화, 최대/최소 정규화+다항식 특성, 로그변환
scale_methods=[(None, None), ('Standard', None), ('Standard', 2),
                ('MinMax', None), ('MinMax', 2), ('Log', None)]
for scale_method in scale_methods:
    X_data_scaled = get_scaled_data(method=scale_method[0], p_degree=scale_method[1],
                                    input_data=X_data)
    print('\n## 변환 유형:{0}, Polynomial Degree:{1}'.format(scale_method[0], scale_method[1]))
    get_linear_reg_eval('Ridge', params=alphas, X_data_n=X_data_scaled,
                        y_target_n=y_target, verbose=False)
```

```
## 변환 유형:None, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.788
alpha 1일 때 5 폴드 세트의 평균 RMSE: 5.653
alpha 10일 때 5 폴드 세트의 평균 RMSE: 5.518
alpha 100일 때 5 폴드 세트의 평균 RMSE: 5.330
```

```
## 변환 유형:Standard, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.826
alpha 1일 때 5 폴드 세트의 평균 RMSE: 5.803
alpha 10일 때 5 폴드 세트의 평균 RMSE: 5.637
alpha 100일 때 5 폴드 세트의 평균 RMSE: 5.421
```

```
## 변환 유형:Standard, Polynomial Degree:2
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 8.827
alpha 1일 때 5 폴드 세트의 평균 RMSE: 6.871
alpha 10일 때 5 폴드 세트의 평균 RMSE: 5.485
alpha 100일 때 5 폴드 세트의 평균 RMSE: 4.634
```

```
## 변환 유형:MinMax, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.764
alpha 1일 때 5 폴드 세트의 평균 RMSE: 5.465
alpha 10일 때 5 폴드 세트의 평균 RMSE: 5.754
alpha 100일 때 5 폴드 세트의 평균 RMSE: 7.635
```

```
## 변환 유형:MinMax, Polynomial Degree:2
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 5.298
alpha 1일 때 5 폴드 세트의 평균 RMSE: 4.323
alpha 10일 때 5 폴드 세트의 평균 RMSE: 5.185
alpha 100일 때 5 폴드 세트의 평균 RMSE: 6.538
```

```
## 변환 유형:Log, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE: 4.770
alpha 1일 때 5 폴드 세트의 평균 RMSE: 4.676
alpha 10일 때 5 폴드 세트의 평균 RMSE: 4.836
alpha 100일 때 5 폴드 세트의 평균 RMSE: 6.241
```

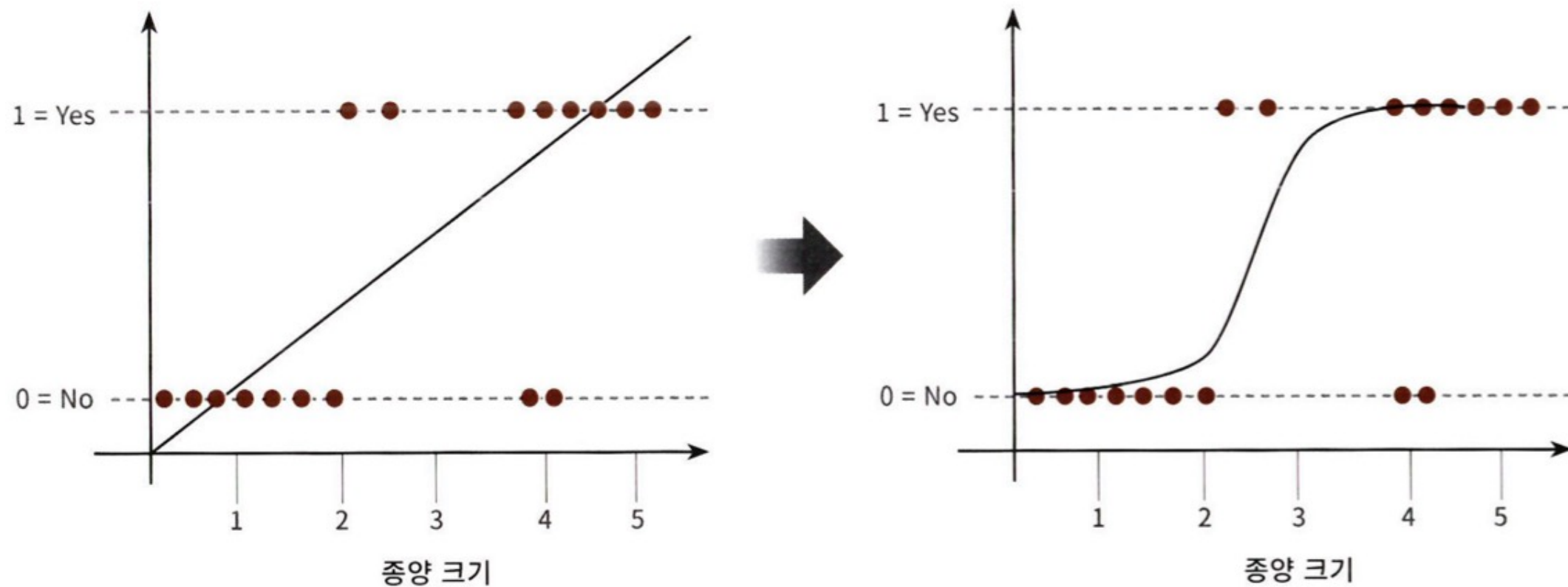


5.7 로지스틱 회귀

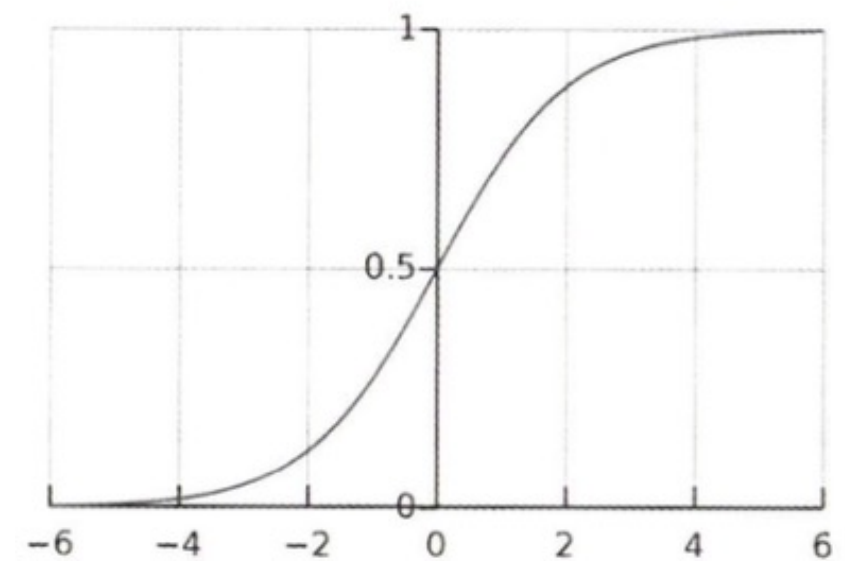


#5.7 로지스틱 회귀

* 로지스틱 회귀 : 선형 회귀 방식을 **분류**에 적용한 알고리즘 >> **분류**에 사용됨



선형 회귀 방식 vs 로지스틱 회귀
>> 시그모이드 함수로 **분류** 수행



〈로지스틱 회귀의 시그모이드 함수〉

*시그모이드 함수 : x값이 아무리 작거나 커져도 Y값은 0과 1에 근사 $y = \frac{1}{1 + e^{-x}}$

#5.7 로지스틱 회귀

```
scaler=StandardScaler()  
data_scaled=scaler.fit_transform(cancer.data)  
X_train,X_test,y_train,y_test=train_test_split(data_scaled,cancer.target,test_size=0.3,random_state=0)
```

데이터에 먼저 정규 분포형태의 표준
스케일링 적용

```
from sklearn.metrics import accuracy_score,roc_auc_score  
# 로지스틱 회귀로 학습 및 예측 수행  
lr_clf=LogisticRegression()  
lr_clf.fit(X_train,y_train)  
lr_preds=lr_clf.predict(X_test)  
  
# 정확도와 ROC-AUC 값 구하기  
print('accuracy:{:0.3f}'.format(accuracy_score(y_test,lr_preds)))  
print('roc_auc:{:0.3f}'.format(roc_auc_score(y_test,lr_preds)))
```

```
accuracy:0.977  
roc_auc:0.972
```

#5.7 로지스틱 회귀

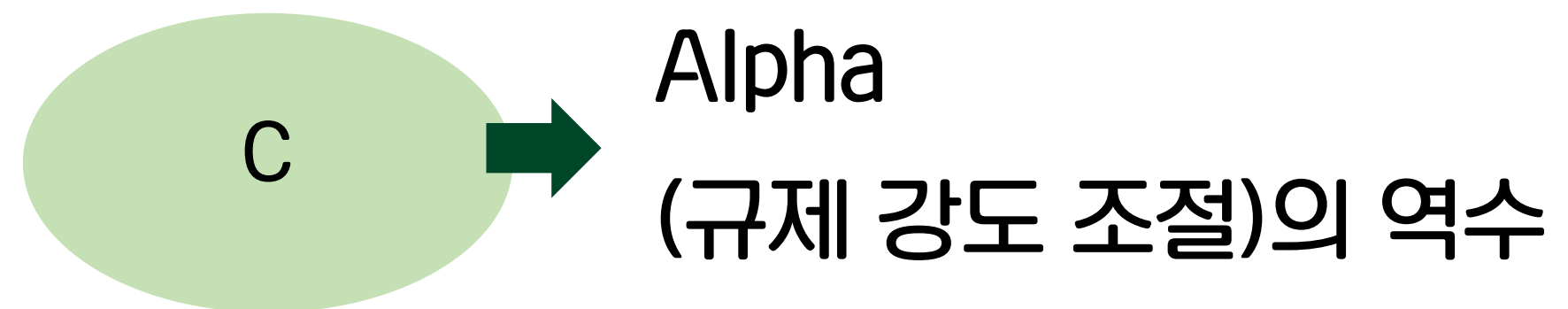
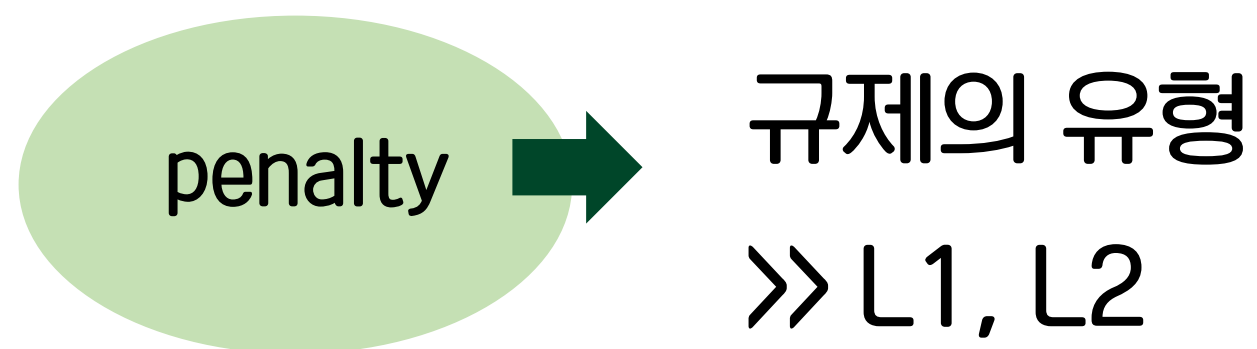
```
from sklearn.model_selection import GridSearchCV
import warnings
warnings.filterwarnings(action='ignore')

params={'penalty':['l2', 'l1'],
        'C':[0.01, 0.1, 1, 1, 5, 10]}

grid_clf = GridSearchCV(lr_clf, param_grid=params, scoring='accuracy', cv=3 )
grid_clf.fit(data_scaled, cancer.target)
print('최적 하이퍼 파라미터:{0}, 최적 평균 정확도:{1:.3f}'.format(grid_clf.best_params_,
                                                                grid_clf.best_score_))
```

최적 하이퍼 파라미터: {'C': 1, 'penalty': 'l2'}, 최적 평균 정확도: 0.975

* 하이퍼 파라미터 적용



5.8 회귀 트리



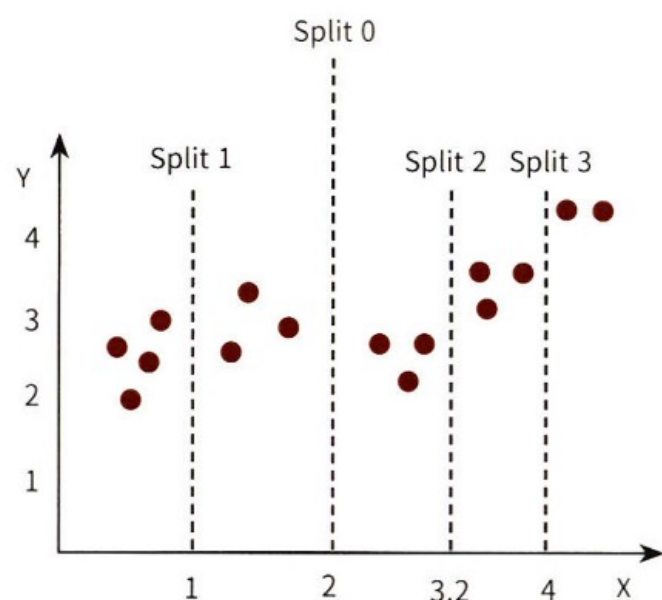
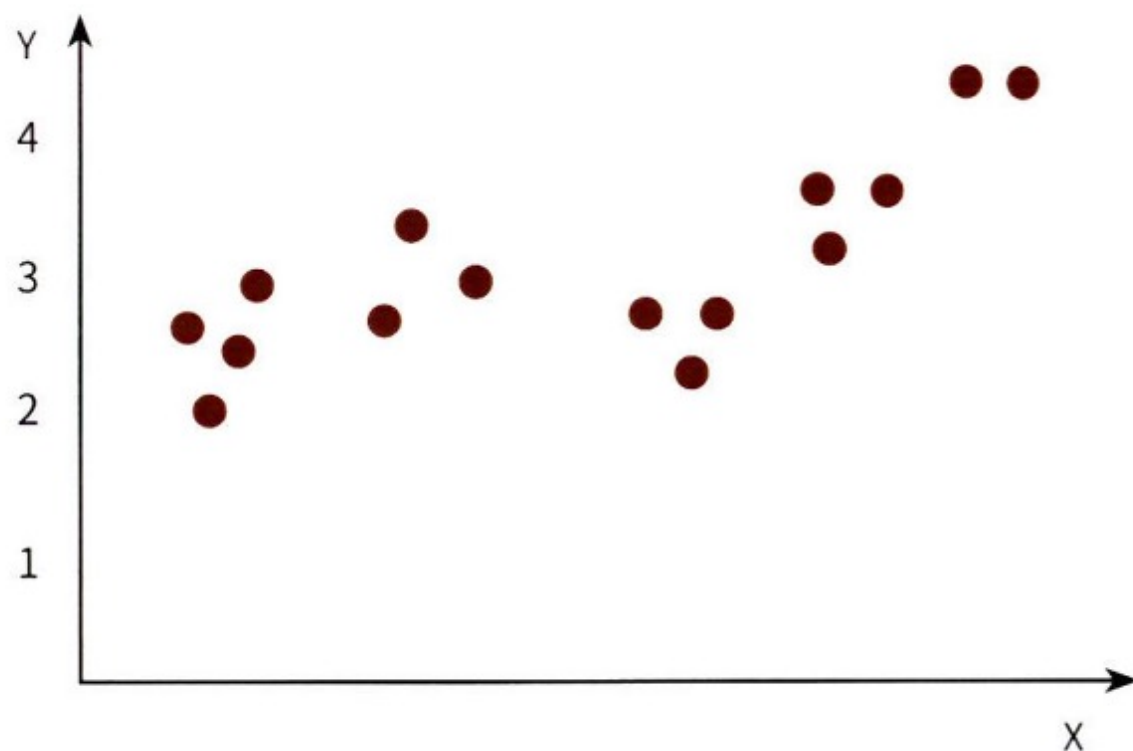
#5.8 회귀 트리

회귀 트리 : 회귀를 위한 트리를 생성하고 이를 기반으로 회귀 예측을 하는 것

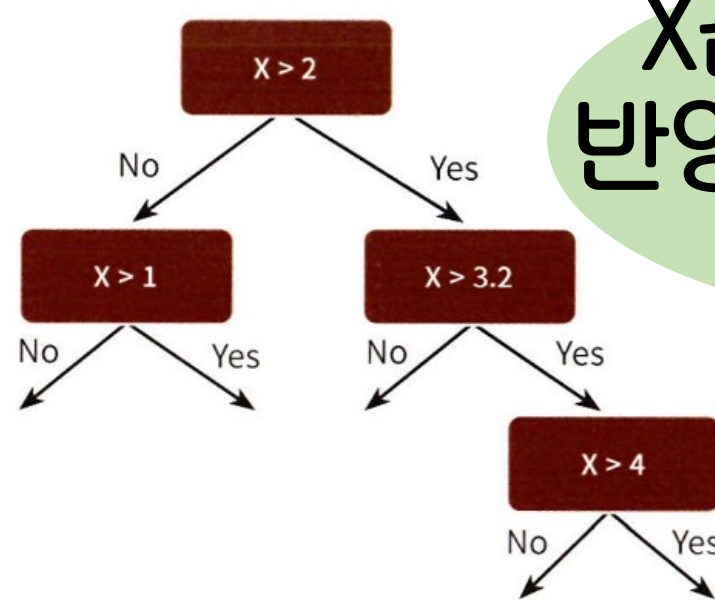
>> 리프노드에 속한 데이터값의 평균값을 구해 회귀 예측값을 계산

>> 비선형 회귀 분석 가능!

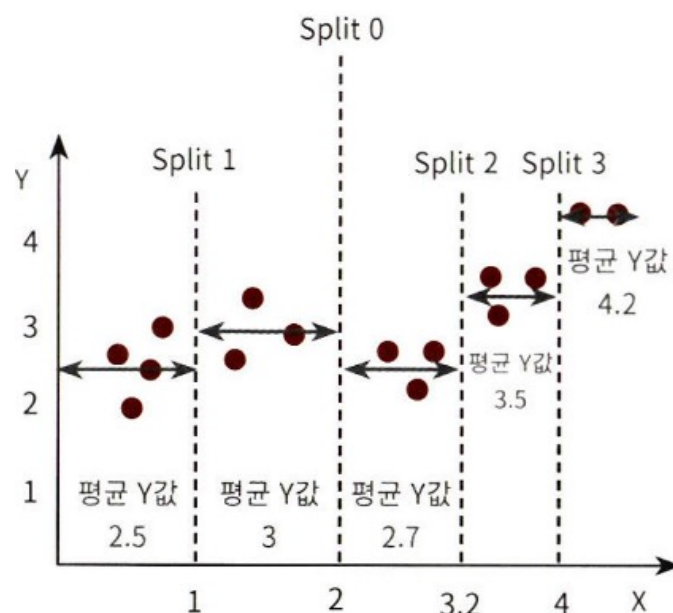
피처가 하나인
데이터 세트



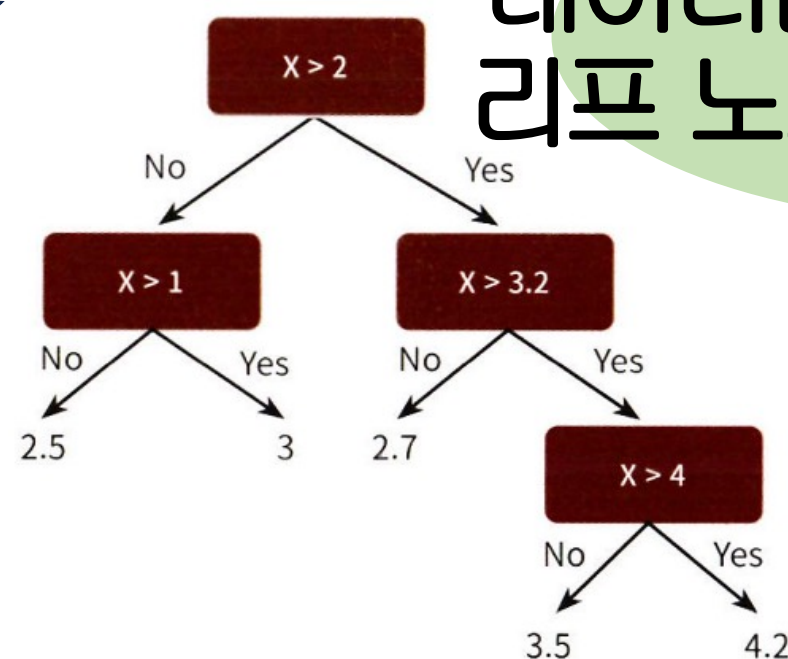
Tree 규칙으로
변환



X값의 균일도를
반영한 지니계수로
분할



평균 Y값으로
예측 값 부여



데이터값의 평균값을
리프 노드 결정값으로
할당

#5.8 회귀 트리

결정 트리, 랜덤 포레스트, GBM XG Boost, LightGBM 등 트리 기반 알고리즘에서 회귀도 가능 >> CART(Classification And Regression Trees)

사이킷런에서 CART 기반 회귀 수행 클래스 제공

알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

#5.8 회귀 트리

랜덤 포레스트 회귀 트리로 예측 수행

```
rf = RandomForestRegressor(random_state=0, n_estimators=1000)
neg_mse_scores = cross_val_score(rf, X_data, y_target, scoring="neg_mean_squared_error", cv = 5)
rmse_scores = np.sqrt(-1 * neg_mse_scores)
avg_rmse = np.mean (rmse_scores)

print('5 folds의 개별 Negative MSE scores :', np.round(neg_mse_scores, 3))
print('5 folds의 개별 RMSE scores :', np.round(rmse_scores, 2))
print('5 folds의 평균 RMSE : {0:.3f}'.format(avg_rmse))
```

```
Boston 데이터 세트 크기: (506, 14)
5 folds의 개별 Negative MSE scores : [ -7.898 -13.027 -20.594 -46.214 -18.841]
5 folds의 개별 RMSE scores : [2.81 3.61 4.54 6.8  4.34]
5 folds의 평균 RMSE : 4.419
```

get_model_cv_prediction() 함수 >> 결정 트리, GBM XG Boost, LightGBM

```
#get_model_cv_prediction() : 입력 모델과 데이터 세트를 입력 받아 교차 검증으로 평균 RMSE를 계산
def get_model_cv_prediction(model, X_data, y_target):
    neg_mse_scores = cross_val_score(model, X_data, y_target, scoring="neg_mean_squared_error",
    rmse_scores = np.sqrt(-1 * neg_mse_scores)
    avg_rmse = np.mean(rmse_scores)
    print('##### ', model.__class__.__name__, ' #####')
    print(' 5 교차 검증의 평균 RMSE : {0:.3f} '.format(avg_rmse))
```

#5.8 회귀 트리

다양한 유형의 회귀 트리로 주택 가격 예측

#다양한 유형의 회귀 트리 생성

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

dt_reg = DecisionTreeRegressor(random_state=0, max_depth=4)
rf_reg = RandomForestRegressor(random_state=0, n_estimators=1000)
gb_reg = GradientBoostingRegressor(random_state=0, n_estimators=1000)
xgb_reg = XGBRegressor(n_estimators=1000)
lgb_reg = LGBMRegressor(n_estimators=1000)
```

트리 기반의 회귀 모델을 반복하면서 평가 수행

```
models = [dt_reg, rf_reg, gb_reg, xgb_reg, lgb_reg]
for model in models:
    get_model_cv_prediction(model, X_data, y_target)
```

```
##### DecisionTreeRegressor #####
5 교차 검증의 평균 RMSE : 5.978
##### RandomForestRegressor #####
5 교차 검증의 평균 RMSE : 4.419
##### GradientBoostingRegressor #####
5 교차 검증의 평균 RMSE : 4.269
##### XGBRegressor #####
5 교차 검증의 평균 RMSE : 4.959
-----
##### LGBMRegressor #####
5 교차 검증의 평균 RMSE : 4.646
```

#5.8 회귀 트리

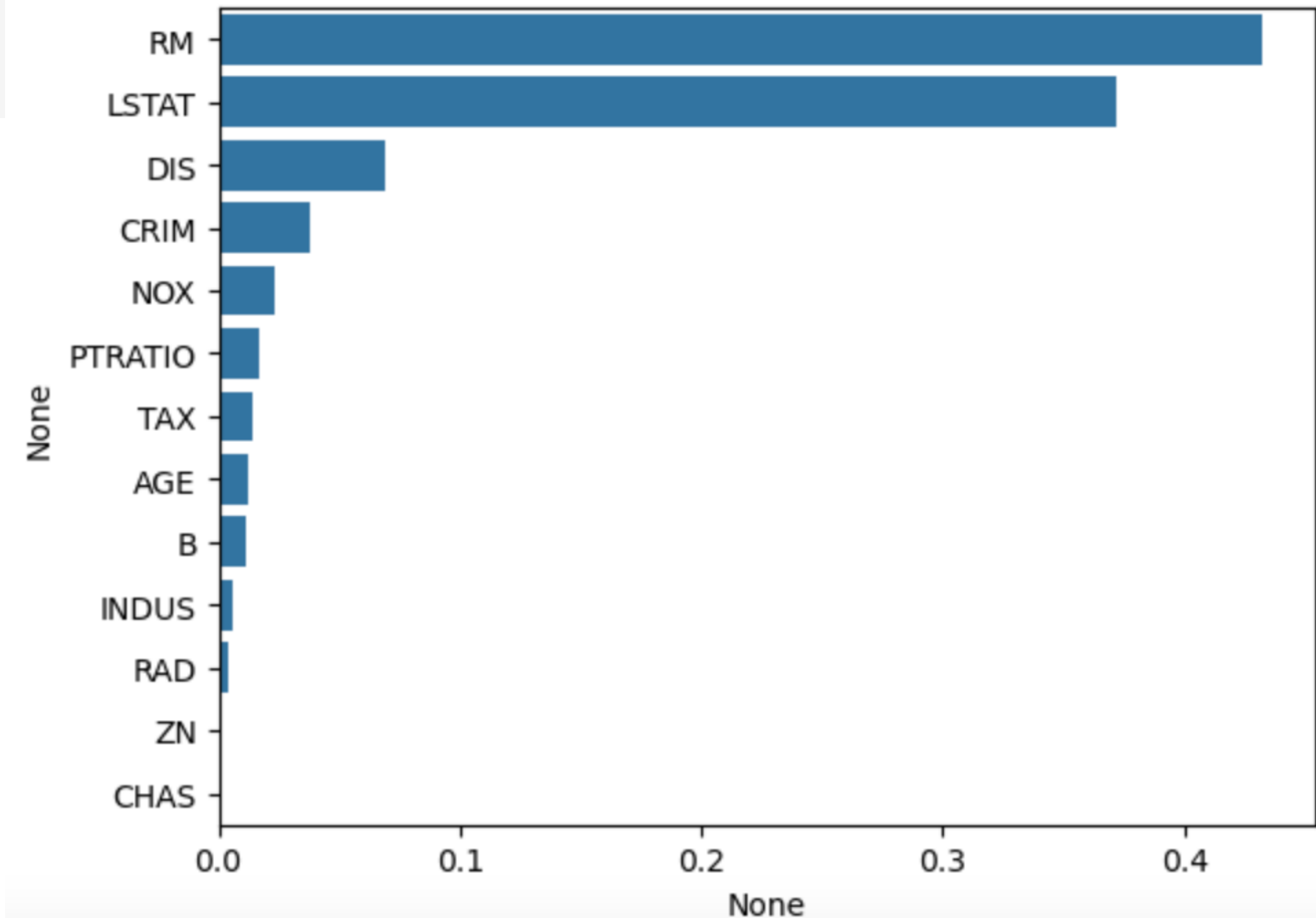
feature_importances_로 중요도 시각화

```
import seaborn as sns
%matplotlib inline

rf_reg = RandomForestRegressor(n_estimators=1000)

# 앞 예제에서 만들어진 X_data, y_target 데이터 셋을 적용하여 학습합니다.
rf_reg.fit(X_data, y_target)

feature_series = pd.Series(data=rf_reg.feature_importances_, index=X_data.columns )
feature_series = feature_series.sort_values(ascending=False)
sns.barplot(x= feature_series, y=feature_series.index)
```

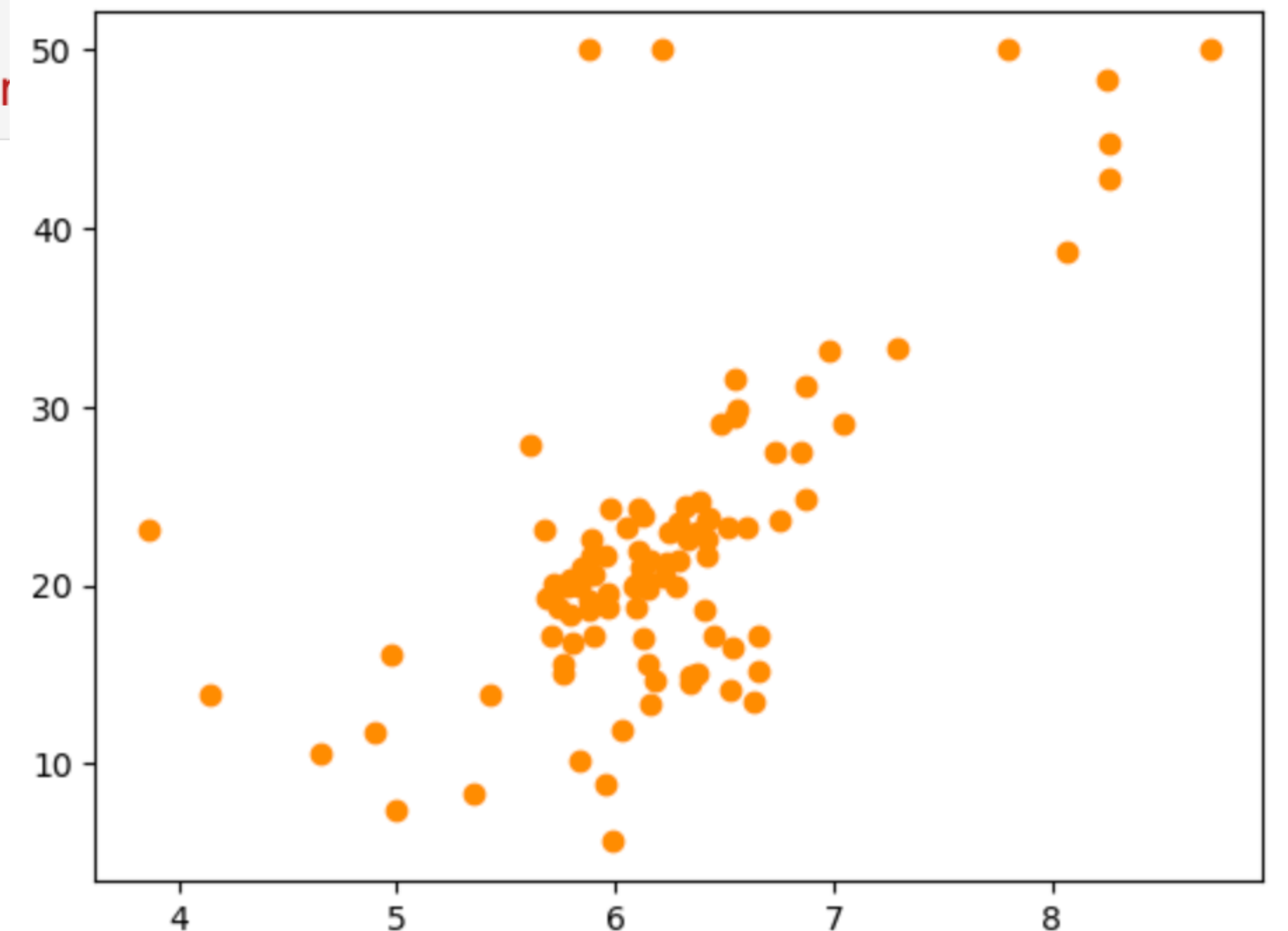


#5.8 회귀 트리

X축에 독립변수인 RM칼럼, Y축에 종속변수 PRICE 칼럼로 예측값을 시각화

```
# 결정 트리의 하이퍼 파라미터인 max_depth의 크기를 변화하여 예측선 확인
import matplotlib.pyplot as plt
%matplotlib inline

bostonDF_sample = bostonDF[['RM', 'PRICE']]
bostonDF_sample = bostonDF_sample.sample(n=100, random_state=0)
print(bostonDF_sample.shape)
plt.figure()
plt.scatter(bostonDF_sample.RM , bostonDF_sample.PRICE, c="darkorange")
```



#5.8 회귀 트리

LinearRegression과 DecisionTreeRegressor를 max_depth를 각각 2,7,로 해서 학습

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 선형 회귀와 결정 트리 기반의 Regressor 생성. DecisionTreeRegressor의 max_depth는 각각 2, 7
lr_reg = LinearRegression() #선형 회귀
rf_reg2 = DecisionTreeRegressor(max_depth=2) #결정 트리 기반 회귀
rf_reg7 = DecisionTreeRegressor(max_depth=7)

# 실제 예측을 적용할 테스트용 데이터 셋을 4.5 ~ 8.5 까지 100개 데이터 셋 생성.
X_test = np.arange(4.5, 8.5, 0.04).reshape(-1, 1)

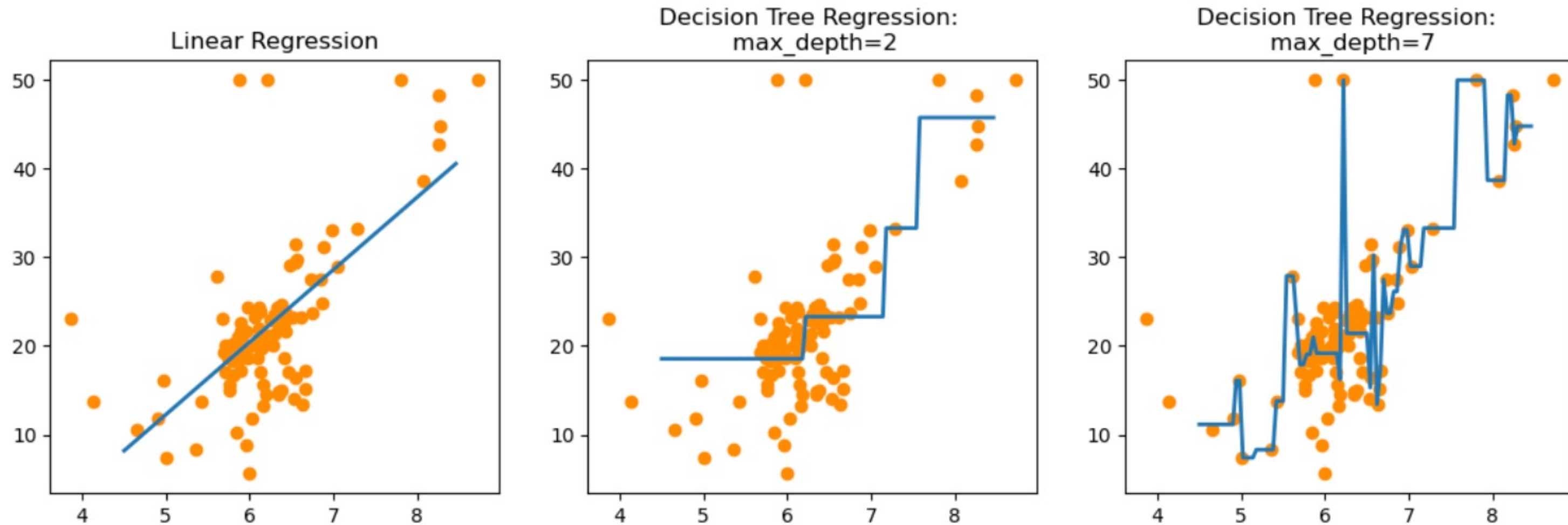
# 보스턴 주택가격 데이터에서 시각화를 위해 피쳐는 RM만, 그리고 결정 데이터인 PRICE 추출
X_feature = bostonDF_sample['RM'].values.reshape(-1,1)
y_target = bostonDF_sample['PRICE'].values.reshape(-1,1)

# 학습과 예측 수행.
lr_reg.fit(X_feature, y_target)
rf_reg2.fit(X_feature, y_target)
rf_reg7.fit(X_feature, y_target)

pred_lr = lr_reg.predict(X_test)
pred_rf2 = rf_reg2.predict(X_test)
pred_rf7 = rf_reg7.predict(X_test)
```

#5.8 회귀 트리

*학습된 Regressor에서 예측한 Price 회귀선



선형 회귀

› 직선으로 표현

회귀 트리(2)

› 계단 형태로

회귀 트리(7)

› 이상치 데이터도 학습
› 과적합 되기 쉬운 모델

Q&A

